OKUVU — Feature Test Register

What must be verified to confirm every functionality works — with each feature clearly attributed to the surface it belongs to.

Version	1.0
Date	17 Sep 2026
Prepared by	oku
Source of truth	code — `OkuvuAI/okuvu` @ `develop` `b4ee3cd` (read-only; repo untouched)
Surface 1	**OKUVU Mobile App** — `apps/mobile` (Flutter client app, for clients)
Surface 2	**OKUVU Therapist Portal** — `apps/therapist-portal` (Next.js + tRPC, for clinicians/admins)

0. Which surface is which

This register covers **two separate products** that talk to the same backend. Every feature below is explicitly marked with the surface it belongs to — look for the **MOBILE APP** or **THERAPIST PORTAL** tag on each heading.

	Mobile App	Therapist Portal
Who uses it	The **client** (Gen Z user)	The **therapist** and **admin**
Code	`apps/mobile` (Flutter, Dart)	`apps/therapist-portal` (Next.js, TypeScript)
Where it runs	iOS / Android app	Web browser
How you get in	Phone OTP (Firebase)	Google OAuth + allowlist
What it does	Chat, journal, breathing, mood, crisis, sessions	Caseload, care plans, notes, homework, reminders, reports, admin
Items in this register	**46** features/behaviours (Part A)	**52** pages/components/procedures (Part B)
Decision rights	Client owns consent + chat-access	Therapist owns clinical config; admin provisions

Rule of thumb: if the client *sees or taps* it → **Mobile App**. If the therapist or admin *sees or taps* it → **Therapist Portal**. The backend (`apps/api` , agents, worker) is **out of scope** here and needs its own register.

1. How to use it

Work the **Test points** under each feature and record evidence (screenshot / log / test name). Re-apply the cross-cutting areas (§2) to **every** item. §5 is the known baseline to start from.

Test levels

Code	Level	What it proves
L1	Static	`flutter analyze` / `tsc` / lint — compiles, no errors
L2	Unit / widget	`flutter test` , `vitest` — logic and component behaviour in isolation
L3	Integration	Flutter `integration_test` / Patrol — multi-screen flows against a real backend
L4	API / contract	Endpoint & tRPC procedure behaviour, auth, validation, error shapes
L5	Manual / device	Human pass on emulator or physical device (UX, feel, gestures)
L6	Safety / clinical	Crisis routing, helplines, consent & visibility enforcement
L7	Non-functional	Performance, accessibility, security, offline resilience

Priority — **P0** safety/privacy/auth/broken core loop (block release) · **P1** core feature degradation (fix before GA) · **P2** polish, edge cases, dev experience.

Status legend (used in §5) — ✅ passing · ❌ failing · ⚠ flaky/partial · ⛔ blocked · ⬜ not built.

---

2. Cross-cutting test areas (re-apply to every feature, on both surfaces)

These are lenses, not features. Each must be verified across the whole surface it applies to.

2.1 Authentication & identity. App: session-gate redirects (`initial → /splash` , `unauthenticated → /auth` , `authenticated → /onboarding/coldstart`); Firebase token on every call; 401 signs out; 503 retries with `Retry-After` . Portal: Google OAuth + allowlist rejection (`/auth?error=unauthorized`); sign-out everywhere; bypass mode divergences.

2.2 Consent & visibility enforcement (do not weaken). Chat-access toggle defaults **off**, is client-owned, and is logged; the portal must never surface summaries/reasoning without it (a known inconsistency exists — §5.4). Partial consent: first run posts 2 rows (safety + chat summaries); the other 3 are just-in-time.

2.3 Safety & crisis. Crisis reachable from **every client screen**; overlay above everything, works offline, back suppressed, `tel:` live; exact helpline strings, 112 present; S3/S4 deterministic routing with no downstream therapeutic step; S4 turns persisted.

2.4 Network, offline & reconnect. Reconnect banner + resume same session; no duplicate turns; offline start, mid-flow drop, slow/ flaky network; optimistic-UI rollback on failed writes.

2.5 Data, privacy & compliance (DPDP). No private data in logs/analytics; journal never read by any agent; phone masked in UI; account deletion incl. 409 `ACTIVE_SESSION` ; clinical access logging on portal reads/writes.

2.6 Accessibility. Tap targets ≥ 44–48px; contrast ≥ 4.5:1; dynamic type; screen-reader labels; no colour-only meaning; sensible focus order.

2.7 Branding & palette. Website palette verbatim; logo on every screen; help-now (crisis) bar on every client screen and deliberately absent on clinician frames. Known drift in code: Material seed still teal, `Inter` / `Fraunces` fonts not bundled.

2.8 Localisation. English default; "Mirror" language mode; Hinglish toggle off; layout survives long strings.

2.9 Performance & resilience. Cold start, transitions, chat streaming latency, long lists; no memory growth; Crashlytics captures fatal errors.

2.10 Security. No secrets in the client bundle; bypass flags inert in prod; portal rate limiting, authorization and assignment checks on every procedure; no IDOR.

---

3. Surface index — every feature, by surface

Use this as the master mapping. It is reproduced in full inside each part.

3.1 MOBILE APP — 46 features

#	Feature	Primary file
#	Feature	Primary file
---	---	---
1	Splash screen	`lib/src/features/splash/splash_screen.dart`
2	OTP login	`lib/src/features/auth/otp_login_screen.dart`
3	Consent screen	`lib/src/features/consent/consent_screen.dart`, model `consent_request.dart`
4	Cold-start screen	`lib/src/features/intake/cold_start_screen.dart`
5	Intake / onboarding chat (cold-start questionnaire)	`lib/src/features/intake/intake_screen.dart`, `intake_notifier.dart`, `intake_message.dart`, `onboarding_provider.dart`, `widgets/onboarding_progress_bar.dart`
6	Home screen	`lib/src/features/home/home_screen.dart`
7	Mood check-in card	`lib/src/features/home/widgets/mood_check_in.dart` (+ provider in `progress/providers/progress_provider.dart`)`
8	Today card	`lib/src/features/home/widgets/today_card.dart`, `providers/today_card_provider.dart`
9	Quick-access tiles (Breathe / Journal)	`lib/src/features/home/widgets/quick_access_tile.dart` (used in `home_screen.dart`)`
10	Progress summary card (Home)	`lib/src/features/home/widgets/progress_summary.dart`
11	Home insight card	`lib/src/features/home/widgets/home_insight_card.dart`
12	Affirmation card	`lib/src/features/home/widgets/affirmation_card.dart`
13	Home bottom navigation bar	`lib/src/features/home/widgets/home_bottom_nav.dart`
14	Breathing screen (quick access + shared tool card)	`lib/src/features/home/breathing_screen.dart`
15	Chat screen (core conversation)	`lib/src/features/chat/chat_screen.dart`, models `chat_message.dart`, `chat_state.dart`, `agent_turn_frame.dart`
16	Typing indicator	`lib/src/features/chat/widgets/typing_indicator.dart`
17	Reconnect banner	`lib/src/features/chat/widgets/reconnect_banner.dart` (also used in intake via an adapter)`
18	Tool cards — dispatcher	`lib/src/features/chat/widgets/tool_card_widget.dart`, model `models/tool_card_config.dart`
19	Guided breathing tool card	`lib/src/features/chat/widgets/guided_breathing_card.dart`
20	Grounding card (interactive 5-4-3-2-1)	`lib/src/features/chat/widgets/grounding_card.dart` (+ unused static variant in `tool_card_widget.dart`)`
21	SRS modal (session-rating prompt)	`lib/src/features/chat/widgets/srs_modal.dart`
22	Voice toggle	`lib/src/widgets/voice_toggle.dart`

#	Feature	Primary file
23	Journal entry form	`lib/src/features/journal/journal_screen.dart`, `journal_provider.dart`, `journal_entry.dart`
24	Journal privacy notice	`lib/src/features/journal/journal_screen.dart`
25	Journal history list	`lib/src/features/journal/journal_list_screen.dart`, `journal_provider.dart`, `journal_entry.dart`
26	Progress screen (mood chart + care phase)	`lib/src/features/progress/progress_screen.dart`, `providers/progress_provider.dart`, `models/mood_data_point.dart`
27	Settings hub	`lib/src/features/settings/settings_screen.dart`
28	My Focus Area	`lib/src/features/settings/my_focus_area_screen.dart`
29	Emergency Contact	`lib/src/features/settings/emergency_contact_screen.dart`, `emergency_contact_notifier.dart`
30	Privacy & Data (consent preferences)	`lib/src/features/settings/privacy_data_screen.dart`, `consent_provider.dart`
31	Language settings (English	Mirror)
32	About	`lib/src/features/settings/about_screen.dart`
33	Privacy Policy viewer	`lib/src/features/settings/privacy_screen.dart`
34	Profile screen (stub)	`lib/src/features/settings/profile_screen.dart`
35	Notifications screen (stub)	`lib/src/features/settings/notifications_screen.dart`
36	Account deletion	`lib/src/features/settings/account_deletion_screen.dart`
37	Quote-of-the-Day toggle	`lib/src/widgets/quote_of_day_toggle.dart`, `features/settings/quote_of_day_notifier.dart`
38	Crisis overlay	`lib/src/features/crisis/crisis_overlay.dart`
39	Not-found screen	`lib/src/core/not_found_screen.dart`
40	TTS / Voice subsystem	`lib/src/core/tts_service.dart`, `lib/src/widgets/voice_toggle.dart`, `lib/src/features/settings/voice_settings_notifier.dart`, plus the independent `FlutterTts` in `guided_breathing_card.`
41	Language mode (English	Mirror) — data path
42	Theme / branding	`lib/src/app.dart`, plus hard-coded `0xFF1A6B5A` across features`
43	WebSocket protocol (shared contract)	`lib/src/core/websocket_service.dart`, `websocket_constants.dart`
44	API surface (all HTTP endpoints found)	—
45	Local storage keys	—
46	Firebase / platform integrations	—

3.2 THERAPIST PORTAL — 52 items

ID	Item	Kind
ID	Item	Kind
---	---	---
B1	Root layout	Infrastructure
B2	Root redirect	Infrastructure
B3	Auth page	Infrastructure
B4	Auth middleware	Infrastructure
B5	Production bypass boot guard	Infrastructure
B6	NextAuth config + allowlist + `auth()` / `signIn` / `signOut`	Infrastructure
B7	Session provider	Infrastructure
B8	tRPC React provider/client	Infrastructure
B9	Router composition	Infrastructure
B10	tRPC context, middlewares, auth/assignment helpers	Infrastructure
B11	In-memory rate limiter	Infrastructure
B12	Server caller factory	Infrastructure
B13	tRPC HTTP handler	Infrastructure
B14	NextAuth route handler	Infrastructure
B15	`format.ts` helpers	Infrastructure
B16	Prisma singleton	Infrastructure
B17	Seed scripts	Infrastructure
C1	Dashboard page	Page / component
C2	Dashboard content	Page / component
C3	Client table	Page / component
C4	Authenticated shell	Page / component
C5	Sidebar	Page / component
C6	Top bar	Page / component
C7	Client page header (unused)	Page / component
C8	Admin page	Page / component
C9	Admin content	Page / component
C10	Client layout	Page / component
C11	Client overview page	Page / component
C12	Program / care plan page	Page / component
C13	Session notes page	Page / component

ID	Item	Kind
C14	Homework page	Page / component
C15	Reminders page	Page / component
C16	Report / Session Prep page	Page / component
D1	`clients.list` — query	tRPC procedure
D2	`clients.detail` — query	tRPC procedure
D3	`clients.getCarePlan` — query	tRPC procedure
D4	`clients.setCarePlan` — mutation	tRPC procedure
D5	`notes.addNote` — mutation	tRPC procedure
D6	`notes.listNotes` — query	tRPC procedure
D7	`homework.assign` — mutation	tRPC procedure
D8	`homework.listHomework` — query	tRPC procedure
D9	`homework.markCompleted` — mutation	tRPC procedure
D10	`reminders.listReminders` — query	tRPC procedure
D11	`reminders.listTools` — query	tRPC procedure
D12	`reminders.create` — mutation	tRPC procedure
D13	`reminders.setActive` — mutation	tRPC procedure
D14	`reportCard.sessionPrep` — query	tRPC procedure
D15	`reportCard.reportCard` — query	tRPC procedure
D16	`admin.amIAdmin` — query	tRPC procedure
D17	`admin.listTherapists` — query	tRPC procedure
D18	`admin.addTherapist` — mutation	tRPC procedure
D19	`admin.assignClient` — mutation	tRPC procedure

Part A — OKUVU MOBILE APP

The Flutter client app in full: **46** feature/behaviour blocks. Every heading in Part A is tagged **MOBILE APP**. This product is the one the **client** uses.

Part B — OKUVU THERAPIST PORTAL

The Next.js clinician/admin portal in full: **31** page/component/infrastructure blocks + **19** tRPC procedures. Every heading in Part B is tagged **THERAPIST PORTAL**. This product is the one the **therapist and admin** use.

Source: `/home/shubham/okuvu-repo/apps/mobile` on branch `develop` (read-only; nothing in the repo modified). App version (pubspec): **1.0.4+5** · package name `okuvu` · Android appId `ai.okuvu.app` (+ `.dev` / `.beta`) · min SDK from Flutter defaults. Stack: Flutter 3.x, `flutter_riverpod` 3.x (Notifier API), `go_router` 17.x, Firebase (core/auth/crashlytics/analytics), `dio`, `web_socket_channel`, `connectivity_plus`, `url_launcher`, `fl_chart`, `shared_preferences`, `flutter_tts`.

> Scope note: this documents only what is present in code. Several places where the task brief or the e2e tests expect behaviour that does **not** exist in `lib/` are flagged under **Risks/unknowns**. All 100 `.dart` files under `lib/` were read.

Cross-cutting architecture (applies to every screen)

- **Entry point** — `lib/main.dart`: `runZonedGuarded` → `AppConfig.init()` → `Firebase.initializeApp()` → Crashlytics error handlers → `runApp(ProviderScope(child: App()))`. Any failure in the first two steps renders a fatal `_FatalErrorApp` (teal `0xFF1A6B5A`, "The app failed to start." + stack).
- **`AppConfig`** — `lib/src/config/app_config.dart`: requires `--dart-define=FLAVOR=dev|beta|prod` (throws `FlavorNotSpecifiedException` otherwise). Optional `BASE_URL` / `WS_URL` overrides. `privacyPolicyUrl` is a hard-coded Google-Docs pub link.
- **Environments** — `lib/src/config/environment.dart`: dev = `http://10.0.2.2:8080` / `ws://10.0.2.2:8080`; beta & prod = `https://backend-971397352159.asia-south1.run.app` / `wss://...`.
- **Dev auth bypass** — `lib/src/config/flavor_config.dart`: `LOCAL_DEV_BYPASS_AUTH` dart-define used only when flavor == dev.
- **Router & guards** — `lib/src/router/router.dart`: routes and redirect table (below). `_RouterNotifier` refreshes on `sessionProvider` change. `_CrisisAwareBackButtonDispatcher` swallows the system back while the crisis overlay is up. `errorBuilder` → `NotFoundScreen`.
- **Session lifecycle** — `lib/src/core/providers.dart`: `sessionProvider` derives `SessionState{initial,authenticated,unauthenticated}` from `FirebaseAuth.instance.authStateChanges()`. `app.dart` listens for authenticated→unauthenticated and then calls `webSocketServiceProvider.disconnect()` and invalidates `chatMessagesProvider`, `emergencyContactProvider`, `voiceSettingsProvider`, `intakeProvider`, `crisisOverlayProvider`, `consentProvider`, `hasCompletedOnboardingProvider`.
- **HTTP** — `lib/src/core/dio_client.dart`: 10 s connect/receive timeouts, JSON headers, `baseUrl = AppConfig.apiBaseUrl`. `AuthInterceptor` (`lib/src/core/auth_interceptor.dart`) attaches `Authorization: Bearer <dev bypass | Firebase idToken>`; on **401** signs the user out; on **503** retries up to **3** times using the `Retry-After` header (default 2 s backoff). Connection errors are only logged.
- **WebSocket** — `lib/src/core/websocket_service.dart`, constants in `websocket_constants.dart` (`sessionPath = '/ws'`). Frames are documented in the Chat section.
- **Theme/branding** — `app.dart`: `colorSchemeSeed: Colors.teal`, Material 3. Hard-coded brand teal `0xFF1A6B5A` recurs across screens. `fontFamily: 'Inter'` is referenced in splash/fatal screen but **no fonts are declared in pubspec** (no `fonts:` section) → falls back to platform default.
- **Analytics** — `lib/src/core/analytics_service.dart` wraps `FirebaseAnalytics.logEvent`, swallowing all errors. Only wired into the SRS modal. Crashlytics records fatal Flutter/platform errors.

Full route table (`lib/src/router/router.dart`)

Part A — OKUVU MOBILE APP (Flutter client app)

Route	Screen
`/splash` (initial)	`SplashScreen`
`/auth`	`OtpLoginScreen`
`/onboarding/consent`	`ConsentScreen`
`/intake`	`IntakeScreen`
`/onboarding/coldstart`	`ColdStartScreen`
`/home`	`HomeScreen`
`/journal`	`JournalScreen`
`/journal/history`	`JournalListScreen`
`/chat`	`ChatScreen`
`/progress`	`ProgressScreen`
`/settings`	`SettingsScreen`
`/settings/profile`	`ProfileScreen` (stub)
`/settings/notifications`	`NotificationsScreen` (stub)
`/settings/privacy`	`PrivacyDataScreen`
`/settings/emergency`	`EmergencyContactScreen`
`/settings/about`	`AboutScreen`
`/settings/privacy-policy`	`PrivacyScreen`
`/settings/delete-account`	`AccountDeletionScreen`
`/settings/focus`	`MyFocusAreaScreen`
`/settings/language`	`LanguageScreen`
(unknown)	`NotFoundScreen`

Redirect rules (session-gated)

- `initial` → force `/splash`.
- `unauthenticated` → force `/auth`.
- `authenticated` and location is `/auth` or `/splash` → `/onboarding/coldstart`.
- Everything else passes through (so `/home`, `/chat` etc. are directly reachable once authenticated; onboarding completion is a client-only flag, see #4/#5).

MOBILE APP 1. Splash screen

File: `lib/src/features/splash/splash_screen.dart` · **Route:** `/splash`

User-visible behaviour Branded teal full-screen splash: "OKUVU" wordmark fades in, then an italic tagline "A space that listens." Both animate over 2.5 s `AnimationController` with staggered intervals. It does not navigate

on a timer; it waits for the session to resolve and then routes to `/auth` (unauthenticated) or `/onboarding/coldstart` (authenticated).

Test points - Verify wordmark/tagline fade animation runs (~2.5 s) and both are eventually fully opaque. - Unauthenticated session → lands on the Sign In screen. - Authenticated session → lands on cold-start (or onward). - `SessionState.initial` (auth still loading) keeps the user on splash (no premature navigation). - Status-bar icons forced light. - Back button cannot return to splash after routing. - No crash if session never resolves (stays on splash).

Dependencies: `sessionProvider` (Firebase auth stream). No API/WS calls. **Risks/unknowns:** none functional; logo is a `TODO` text placeholder (no asset).

MOBILE APP

2. OTP login

File: `lib/src/features/auth/otp_login_screen.dart` · **Route:** `/auth`

User-visible behaviour Two-step phone login. Step 1: a "Sign In" app bar, a phone icon, "+91" prefix, a phone-number field (`ValueKey('phone_input')` , hint `99999 99999`) and a full-width "Continue with Phone" button. On submit, `FirebaseAuth.verifyPhoneNumber` fires and the screen advances to step 2: six single-character OTP boxes (`ValueKey('otp_0')` … `otp_5`) with auto-advance and auto-submit on the last digit, a "Verify & Continue" button, and a 30-second resend cooldown that becomes a "Resend code" text button. A back arrow returns to the phone step.

Test points - Phone validation: accepts `^(\+91)?[6-9]\d{9}$` after stripping spaces/hyphens; rejects short/long/non-6–9-leading numbers with "Enter a valid 10-digit Indian phone number". - Number normalised to `+91` + last 10 digits before sending to Firebase. - "Continue with Phone" shows an inline spinner while `_isSending` and is disabled during send. - Firebase `onError` (e.g. invalid number/quota) surfaces `e.message` as `_phoneError` . - OTP entry: typing advances focus; backspace on an empty box moves focus back; filling the 6th box auto-triggers `_verifyOtp()` . - "Verify & Continue" visible but disabled-overlayed while verifying (spinner overlays it). - Wrong OTP → "Incorrect code. Try again." and fields cleared + focus reset. - Resend cooldown counts down 30→0, shows "Resend code in N s", then a tappable "Resend code"; resend clears OTP fields and restarts the cooldown. - Back arrow from OTP step returns to phone step, cancels the timer, resets error/verification. - Network/ `DioException` on `POST /auth/verify-otp` → "Something went wrong. Please try again." - On successful Firebase sign-in + backend `is_new_user != false` → navigate to `/onboarding/coldstart` . - Empty/short OTP does not submit. - Text/max-length: each OTP box `maxLength: 1` , numeric keyboard.

Dependencies: Firebase Auth (`AuthService.verifyPhone` / `verifyOtp`), `dioClientProvider` , `POST /auth/verify-otp {phone_number}` , `routerProvider` . **Risks/unknowns** - If the backend returns `is_new_user == false` (returning user), the code does **not** navigate anywhere — user could remain stuck on the OTP screen (only Firebase sign-in side-effects occur). No branch handles existing users explicitly. - Response-shape guess: `data['is_new_user']` ; if `data` is not a Map, treated as onboarding. - No Firebase test-phone hint in UI; e2e relies on `E2E_TEST_PHONE` / `E2E_TEST_OTP` dart-defines.

MOBILE APP

3. Consent screen

File: `lib/src/features/consent/consent_screen.dart` , model `consent_request.dart` · **Route:** `/onboarding/consent`

User-visible behaviour "Step 1 of 8" onboarding screen titled "Before we begin". Four consent cards with checkboxes: **Safety monitoring** (Required, checkbox disabled/always-on), **Mood understanding**, **Progress**

tracking (all default on), and **Emergency contact** (default off). The primary button "Save and continue" stays visually greyed/disabled until the user scrolls the list to the bottom; a white gradient hints at more content. Footer: "You can change any of these in Settings > Privacy." On save it POSTs consent and navigates to `/intake`.

Test points - Safety-monitoring checkbox is non-interactive and always checked; badge "Required" visible. - Button disabled (no-op) until the list is scrolled within 10 px of the bottom; gradient overlay disappears then. - Toggling the three optional cards updates their checkbox state. - Successful `POST /users/me/consent` → navigate `/intake`. - 401 → "Session expired. Please log in again."; 400 → backend `error.message` or generic; other → generic. Button re-enabled after error. - Dev flavor + network error → proceeds to `/intake` (local-only consent fallback). - Error text is rendered above the button. - Back arrow (`context.pop()`) behaviour. - Payload keys exactly `safety_monitoring`, `mood_inference`, `assessment`, `emergency_contact`.

Dependencies: `dioClientProvider`, `POST /users/me/consent`, `AppConfig.flavor`. **Risks/unknowns:** "Step 1 of 8" is hard-coded text (no real multi-step engine); no analytics; scroll-to-bottom gate can be hard for very short content/screen heights.

MOBILE APP 4. Cold-start screen

File: `lib/src/features/intake/cold_start_screen.dart` · **Route:** `/onboarding/coldstart`

User-visible behaviour A transition/"welcome" screen that resolves the user's care plan. While `GET /users/me/context` is in flight it shows a centered spinner. If a care plan exists it shows "Your space is ready.", a custom-painted two-people+heart graphic, explanatory copy ("Your therapist has set up your focus area..."), a card showing **Your focus area** = the mapped plan name, and a "Let's begin" button → `/chat`. If there is **no** care plan, or the request errors, it silently resolves to `/home` (if onboarding already completed) or `/onboarding/consent` (new user).

Test points - Loading → spinner. - Plan present (CP1/CP2/CP3 → "Anxiety & Worry"/"Low Mood"/"Stress & Burnout"; unknown → "General Support") → content + "Let's begin" navigates to `/chat`. - No plan (`care_plan == null`) → routes to consent (fresh) or home (onboarded). - Network error / 3 s timeout → same fallback (never stuck). - Care-plan card shows the mapped friendly name, not the raw id. - Guard `_navigated` prevents double navigation. - Back navigation is not offered (no app bar).

Dependencies: `userCarePlanProvider` (`GET /users/me/context`, 3 s connect/receive timeouts), `hasCompletedOnboardingProvider`. **Risks/unknowns:** `GET /users/me/context` failures are indistinguishable from "no plan" (both → fallback), so a transient outage can send an onboarded user to consent. Expected response shape `{care_plan:{care_plan_id,assignment_state}, engagement:{current_streak,longest_streak}}`.

MOBILE APP 5. Intake / onboarding chat (cold-start questionnaire)

Files: `lib/src/features/intake/intake_screen.dart`, `intake_notifier.dart`, `intake_message.dart`, `onboarding_provider.dart`, `widgets/onboarding_progress_bar.dart` · **Route:** `/intake`

User-visible behaviour A chat-style questionnaire ("OKUVU" app bar, 3-segment progress bar). The app opens a WebSocket session and the agent sends up to three scripted turns; the user types free-text replies in an input bar (`ValueKey('intake_input')` / `ValueKey('intake_send')`). While awaiting a reply a typing indicator shows and the input is hidden. When the flow completes, onboarding is marked complete, the WS is disconnected, and the app navigates to `/home`. A reconnect banner appears if the socket drops. A back arrow (visible once past the opening message and before completion) returns to the consent screen.

Test points - Opening: spinner until the first agent message arrives. - Turn states: input enabled only during `turn{1,2,3}Active`; hidden during `waitingForTurn*Reply` (typing indicator shown instead). - Send button disabled when the field is empty; enabled after typing. - Progress bar fills 1 → 2 → 3 segments (`segmentCount: 3`) as turns progress. - Agent messages render as left bubbles with a green "E" avatar; user messages right-aligned in primary color. - Auto-scroll to newest message. - Reaching `complete` → sets `hasCompletedOnboardingProvider.complete()`, calls WS `disconnect()`, navigates to `/home` (single-fire guard). - `SESSION_CLOSED` from the server also triggers completion. - Reconnect: after a reconnect `sessionActive`, onboarding turn count resets and messages clear (fresh questionnaire). - Back arrow navigates to `/onboarding/consent`. - Reconnect banner appears while the socket is down. - WS connect uses `Environment.fromString(AppConfig.flavor).wsBaseUrl` + Firebase id token.

Dependencies: `intakeProvider`, `webSocketServiceProvider` (`connect`, `sendUserTurn`, `disconnect`), `agentTurnStream`, `sessionClosedStream`, `wsShowBannerProvider`, `hasCompletedOnboardingProvider`, Firebase `getIdToken()`.
Risks/unknowns - **No care-plan selection UI exists** in the intake screen despite the task brief and the e2e tests referencing `ValueKey('care_plan_0')` / "Anxiety support"/"Low Mood" — those e2e flows are **stale** and will fail against current code. - Unit test `test/features/intake/intake_screen_test.dart` expects an initial assistant message "What's been weighing on your mind lately?" and `filledSegments == 2`, neither of which matches the current notifier (starts empty, 0 segments) → also stale. - A mid-questionnaire reconnect wipes answers (no persistence). - Turn counting is client-side only; a mis-ordered/missing server frame can stall the flow.

MOBILE APP 6. Home screen

File: `lib/src/features/home/home_screen.dart` · **Route:** `/home`

User-visible behaviour Greeting header ("Good morning/afternoon/evening" by hour) and a formatted date ("Monday, January 5"). A scrollable stack of cards: mood check-in (#7), Today card (#8), two quick-access tiles (#9), progress summary (#10), a "FROM OKUVU" insight card (#11), a daily affirmation (#12), a red "Crisis Support" text button, and a bottom navigation bar (#13).

Test points - Greeting switches at 12:00 and 17:00 boundaries. - Date string matches current day/month. - All listed cards render in order; bottom nav has 4 destinations. - "Crisis Support" tap opens the crisis overlay (#41). - List scrolls without overflow on small screens. - If `journalListProvider` fails, the Journal tile shows "0 entries" (no crash). - App bar title "OKUVU".

Dependencies: `journalListProvider` (for the entry count), `crisisOverlayProvider`, child widgets/providers.
Risks/unknowns: The insight message is a hard-coded constant ("Your insights will appear here after your first check-in."); `ProgressSummary` is always called with `showData: false`; several cards are placeholders (see individual entries).

MOBILE APP 7. Mood check-in card

File: `lib/src/features/home/widgets/mood_check_in.dart` (+ provider in `progress/providers/progress_provider.dart`)

User-visible behaviour "How are you feeling today?" with five tappable numbered circles (1–5), each styled by `kMoodColors` (`1 Struggling`, `2 Down`, `3 Okay`, `4 Good`, `5 Great`). Tapping a number selects it (scales up, fills with its color) and posts a check-in. Tapping the same number deselects. For mood ≥ 4 it shows "Glad to hear that. Have a good one." and auto-clears after 3 s. For mood ≤ 2 it shows "Want to talk about it?" with a "Check in" button → `/chat` and a "Maybe later" button.

Test points - Each of 1–5 is selectable and shows the correct label/color. - Re-tap clears the selection; selecting another switches. - Mood 4/5 auto-dismisses after ~3 s. - Mood 1/2 shows the talk prompt; "Check in" → `/chat`; "Maybe later" clears. - `POST /progress/mood/checkin {mood_value}` fires on selection; failure shows snackbar "Couldn't save your check-in. Please try again.". - Success invalidates `moodProgressProvider` (progress chart updates). - Re-selects overwrite today's value (IST-day upsert).

Dependencies: `moodCheckinProvider` → `POST /progress/mood/checkin`; `moodProgressProvider`; `context.go('/chat')`. **Risks/unknowns:** Optimistic UI — the circle shows selected even if the POST later fails (label then snackbar). Auto-dismiss timer for high moods isn't cancelled if the widget rebuilds via navigation.

MOBILE APP 8. Today card

Files: `lib/src/features/home/widgets/today_card.dart`, `providers/today_card_provider.dart`

User-visible behaviour A "Today" card that is **hidden entirely** while loading/erroring or when there is no content. When data exists it shows: an optional italic quote; an optional homework line with a "Mark done" button; and a list of reminders (bell icon + `scheduleNote`, optionally with `(toolLabel)`).

Test points - Hidden (zero-height) when `GET /users/me/today` is loading or errors. - Hidden when homework, reminders and quote are all absent. - Quote renders in italics. - "Mark done" → `POST /users/me/homework/:id/complete`; on success the card refreshes (provider invalidated); on failure a snackbar "Couldn't save that. Please try again.". - Reminders render each `scheduleNote`; tool label appended only when present. - Parsing tolerates missing `reminders` / `homework` / `quote` / `homeworkId`.

Dependencies: `todayCardProvider` (`GET /users/me/today`), `homeworkCompleteProvider` (`POST /users/me/homework/:id/complete`). **Risks/unknowns:** JSON keys are camelCase (`scheduleNote`, `toolLabel`) — must match the backend. No loading/empty affordance is ever shown (silent absence).

MOBILE APP 9. Quick-access tiles (Breathe / Journal)

File: `lib/src/features/home/widgets/quick_access_tile.dart` (used in `home_screen.dart`)

User-visible behaviour Two side-by-side tiles. **Breathe** (icon, subtitle "4 min") pushes a `BreathingScreen` via `Navigator.push`. **Journal** (icon, subtitle "N entries") navigates to `/journal`; the count comes from `journalListProvider` (defaults to 0).

Test points - Breathe tile opens the breathing screen (#14) and back returns to home. - Journal tile → `/journal`. - Entry count reflects the journal list length; shows "0 entries" when empty/loading/error.

Dependencies: `journalListProvider`. **Risks/unknowns:** "4 min" is hard-coded; tile is a `GestureDetector` (no Material ink/ripple). No semantic label — accessibility needs checking.

MOBILE APP 10. Progress summary card (Home)

File: `lib/src/features/home/widgets/progress_summary.dart`

User-visible behaviour On Home it is always called with `showData: false`, so it renders the **empty state**: a trending-up icon, "No mood check-ins yet," and "Your progress will appear here after your first check-in." A data

state exists in code (streak/check-in stat tiles + "Gaps happen. You're still here.") but is only reachable if `showData` is true, which nothing currently sets.

Test points - Home always shows the empty-state copy. - `showData: true` path renders the two stat tiles with placeholder values ("3", "12") — reachable only via direct widget test. - No navigation.

Dependencies: none. **Risks/unknowns:** The data state uses hard-coded placeholder values (`TODO(E8-S01)`); on Home it is dead UI. Not backed by backend.

MOBILE APP

11. Home insight card

File: `lib/src/features/home/widgets/home_insight_card.dart`

User-visible behaviour A card headed "FROM OKUVU" with a message string passed in by Home (currently a constant placeholder). Static, non-interactive.

Test points - Header "FROM OKUVU" present; message text renders. - Empty/whitespace message still renders safely.

Dependencies: none (message provided by caller). **Risks/unknowns:** No backend wiring yet on Home (placeholder text only).

MOBILE APP

12. Affirmation card

File: `lib/src/features/home/widgets/affirmation_card.dart`

User-visible behaviour A soft-green card with "□ Today" and a supportive affirmation. The line is chosen deterministically by day index: `now.difference(DateTime(2026,1,1)).inDays % 30` into a 30-item list (index 0 "What you're feeling right now is real…" … 29 "Today, you don't have to solve everything…").

Test points - The affirmation is stable within a day and changes day-to-day. - All 30 strings are well-formed; index never out of range (including dates before 2026-01-01 — modulo of a negative would need checking). - Header and emoji render. - No interaction.

Dependencies: none (purely local). **Risks/unknowns:** Dates before 2026-01-01 could yield a negative modulo index (potential out-of-range) — worth a boundary test. Not personalised/care-plan-aware.

MOBILE APP

13. Home bottom navigation bar

File: `lib/src/features/home/widgets/home_bottom_nav.dart`

User-visible behaviour Material 3 `NavigationBar` with Home / Chat / Progress / Settings (index 0 selected). The Chat destination shows a small teal badge while `wsConnectionStateProvider == sessionActive && !chatMessagesProvider.isSessionClosed` (hinting an open session).

Test points - Tapping Chat/Progress/Settings navigates to `/chat` / `/progress` / `/settings` ; Home is a no-op. - Chat badge appears only during an active, unclosed WS session; disappears when the session closes. - Labels are always shown.

Dependencies: `wsConnectionStateProvider`, `chatMessagesProvider`. **Risks/unknowns:** Badge visibility is an approximation (the code comments acknowledge there is no dedicated "incomplete session" provider).

MOBILE APP 14. Breathing screen (quick access + shared tool card)

Files: `lib/src/features/home/breathing_screen.dart` · reuses `lib/src/features/chat/widgets/guided_breathing_card.dart` **Route:** pushed via `Navigator` (no named route)

User-visible behaviour An "Breathing Exercise" screen with a back arrow that plays a guided 4-2-6 breathing animation (3 cycles) using the shared `GuidedBreathingCard` (see #19 for mechanics). Reached from the Home "Breathe" tile.

Test points - Back arrow pops to Home. - Card starts in "ready" with Start/Skip; Start runs inhale(4)/hold(2)/exhale(6) × 3 with a growing/shrinking circle and count. - Completion shows "Done. How do you feel now?". - Voice lines play only when voice is enabled and the en-IN TTS pack is available.

Dependencies: `ToolCardConfig(type: guidedBreathing, breathingConfig 4-2-6-3)`, `voiceSettingsStateProvider`, `flutter_tts`. **Risks/unknowns:** TTS language here is `en-IN` (hard-coded), while `TtsService` checks `en-US` — inconsistent language availability gate.

MOBILE APP 15. Chat screen (core conversation)

Files: `lib/src/features/chat/chat_screen.dart`, models `chat_message.dart`, `chat_state.dart`, `agent_turn_frame.dart` · **Route:** `/chat`

User-visible behaviour The main therapy chat. Header shows "OKUVU / Your session" plus a cloud-off icon when offline. A `VoiceToggle` sits at the top. Messages render as left (agent, "E" avatar, white bubble) / right (user, primary-colored bubble) bubbles, capped at 75 % width. A typing indicator shows while awaiting a reply. The input bar (`ValueKey('chat_input')` / `ValueKey('chat_send')`) is disabled while not in `sessionActive`, while the agent is typing, or once the session is closed. Entering `/chat` connects the WebSocket (dev bypass token or Firebase id token). Ending the session swaps the input for "This session has ended." with a "Back to Home" button (`ValueKey('back_to_home')`), and the SRS modal is shown. Back navigation is guarded (`PopScope canPop:false`).

Test points - Empty state "How are you feeling today?" with chat icon before any messages. - Send: requires `sessionActive`; otherwise a snackbar "Not connected. Please wait..." and the text is preserved. - Sending appends the user bubble, clears the field, shows the typing indicator, starts a 20 s timeout, and scrolls to bottom. - 20 s with no agent reply → typing stops + snackbar "No response received. Please try again.". - Incoming `AGENT_TURN` appends an agent bubble (deduped by `turn_id`), stops typing, and invokes TTS. - `SESSION_CLOSED` (reason ≠ `IDLE_TIMEOUT`) → session-closed UI + SRS modal after ~400 ms. - `IDLE_TIMEOUT` close is silent (no closed UI, no SRS) and the service silently reopens a session. - CRISIS_SIGNAL shows the crisis overlay; the chat back button is disabled while the overlay is active. - Back button/PopScope: if session closed → `/home`; else an "End this session?" dialog with Continue/End session; ending disconnects and goes `/home`. - Tool cards render under agent bubbles (see #17/#19). - Journal-prompt suggestion (if `journal_prompt_suggested`) renders a "Save to journal" button that seeds the journal form and navigates to `/journal`. - Reconnect banner overlays the top of the body when the socket drops. - Offline: cloud-off icon in the app bar. - Input disabled whenever the socket isn't `sessionActive`.

Dependencies: `chatServiceProvider` (provider_overrides.dart), `chatServiceProvider.connect/sendUserTurn/disconnect`, `chatMessagesProvider`, `wsConnectionStateProvider`, `isOnlineProvider` (connectivity_plus), `crisisOverlayProvider`, `sessionIdProvider`,

`journalEntryFormProvider`, `authServiceProvider`, `AppConfig.wsBaseUrl`. **Risks/unknowns** - `dispose()` does not disconnect the socket (by design; state persists across screens). - The 20 s typing timeout is a UI-only heuristic; a slow backend turn > 20 s shows a spurious failure snackbar. - `_useMockResponses` is `false` (mock path dead). - The commented-out `/crisis` navigation is dead legacy code (route was never registered).

MOBILE APP 16. Typing indicator

File: `lib/src/features/chat/widgets/typing_indicator.dart`

User-visible behaviour An agent-styled bubble with three dots that bob in a staggered sine animation (1.2 s loop), shown while awaiting a reply (chat and intake).

Test points - Renders three animated dots; animation loops/repeats. - Appears as the last list item while `_isAgentTyping` (chat) or `isWaitingForReply` (intake). - Removed as soon as a reply arrives.

Dependencies: none. **Risks/unknowns:** none functional.

MOBILE APP 17. Reconnect banner

File: `lib/src/features/chat/widgets/reconnect_banner.dart` (also used in intake via an adapter)

User-visible behaviour An animated (size+fade) top banner driven by `wsShowBannerProvider`. When the WS state is `error` it shows a wifi-off icon, "Connection lost", and a "Retry" button that calls `WebSocketService.retry()`. Otherwise (connecting/reconnecting) it shows a spinner and "Reconnecting…".

Test points - Appears ~500 ms after a disconnect (debounce) and hides on `sessionActive`. - In error state: "Connection lost" + Retry; tapping Retry re-attempts the connection. - In non-error state: spinner + "Reconnecting…" and no Retry. - Animates in/out cleanly (no layout jump beyond size transition). - Renders in both chat and intake.

Dependencies: `wsShowBannerProvider`, `wsConnectionStateProvider`, `WebSocketService.retry()`. **Risks/unknowns:** none functional.

MOBILE APP 18. Tool cards — dispatcher

File: `lib/src/features/chat/widgets/tool_card_widget.dart`, model `models/tool_card_config.dart`

User-visible behaviour `ToolCardWidget` maps a `ToolCardConfig.type` to a widget: `guidedBreathing` / `boxBreathing` → `GuidedBreathingCard`; `grounding5431` → a grounding card.

Test points - Correct widget chosen per type; unknown JSON type defaults to `guided_breathing`. - `BreathingConfig` defaults (4/2/6/3, "Breathing Exercise") when fields are missing. - JSON value round-trips (`guided_breathing`, `box_breathing`, `grounding_5431`).

Dependencies: `AgentTurnFrame.payload.tool_card`. **Risks/unknowns — IMPORTANT:** `tool_card_widget.dart` declares its **own** class named `GroundingCard` (a static display card) while also importing `grounding_card.dart` which declares an interactive `GroundingCard`. In Dart the local declaration shadows the import, so the `case grounding5431` returns the **static** card. The interactive grounding experience (which collects user input and calls `sendToolResult`) is therefore **not** what chat shows. See #20.

MOBILE APP 19. Guided breathing tool card

File: `lib/src/features/chat/widgets/guided_breathing_card.dart`

User-visible behaviour An in-chat card (also reused by the Breathing screen) with a pulsing circle. "Start" runs the configured cycles, speaking "Breathe in/Hold/Breathe out" and each count when voice is on, animating the circle up/down; "Stop"/"Skip" aborts. On completion: "Done. How do you feel now?". Shows the cycle counter and the ratio label (e.g. `4-2-6`).

Test points - Ready state has Start + Skip; active state has Stop and a `current/cycles` counter; complete state shows the done message. - Circle expands on inhale and contracts on exhale; a per-second count displays. - Hold phase is skipped when `holdSeconds == 0`. - Voice lines only when voice is enabled; each count is spoken. - Skip/Stop halts both TTS and animation and moves to complete. - Dispose stops TTS. - `box_breathing` renders via the same widget.

Dependencies: `voiceSettingsStateProvider`, `flutter_tts` (en-IN), `ToolCardConfig.breathingConfig`. **Risks/unknowns:** The card creates its **own** `FlutterTts` instance (independent of `TtsService`), so the "voice enabled" setting is read but the shared TTS service state isn't; en-IN pack may be missing on some devices (fails silently).

MOBILE APP 20. Grounding card (interactive 5-4-3-2-1)

File: `lib/src/features/chat/widgets/grounding_card.dart` (+ unused static variant in `tool_card_widget.dart`)

User-visible behaviour A step-through 5-4-3-2-1 exercise: for each sense (5 see, 4 touch, 3 hear, 2 smell, 1 taste) the user types items, which appear as pills; progress dots advance as each sense is satisfied; a "Skip" button jumps to the end. On finish it POSTs a `TOOL_RESULT` over the WebSocket and collapses to "Grounding exercise complete" (or "...skipped"). Header "5-4-3-2-1 Grounding", per-step `count/required` counter.

Test points - Typing adds a pill; the counter increments; when a step reaches its required count it advances to the next sense (and focuses the field); the last sense completes the exercise. - Empty input is ignored. - Skip collapses to the skipped state. - Completing sends a `TOOL_RESULT` frame with `tool_type: grounding_5431` and `result` keyed `see/touch/hear/smell/taste`. - Progress dots reflect completed/current/upcoming. - The check icon shows on the final item of a sense.

Dependencies: `webSocketServiceProvider.sendToolResult` → `TOOL_RESULT` frame; `GroundingResult.toJson`. **Risks/unknowns — IMPORTANT:** As noted in #18, this interactive card is **shadowed** by the identically-named static `GroundingCard` in `tool_card_widget.dart`, so chat currently renders the static version and the interactive flow (and its `TOOL_RESULT`) is not reachable through the normal chat path. A widget/unit test imports this file directly, so tests can exercise it while the running app does not.

MOBILE APP 21. SRS modal (session-rating prompt)

File: `lib/src/features/chat/widgets/srs_modal.dart`

User-visible behaviour A bottom sheet titled "How was this session?" with subtitle "Optional — takes 2 seconds." and four icon options — "Missed the mark", "Not really", "Okay", "Helpful" (scores 1–4) — plus a "Skip" action. Selecting an option dims the others, shows a check confirmation ("Got it") and auto-dismisses after ~800 ms. Shown when a session closes.

Test points - Trigger: session closes (from chat) or "Back to Home" while not yet rated. - Tapping an option records `srs_rating_selected {score,label}` and, if a `sessionId` exists, `PATCH /sessions/:id/srs {srs_score}`; no session id → PATCH skipped (logged). - "Skip" logs `srs_skipped` and dismisses. - Confirmation state shows for ~800 ms then auto-dismisses; `srs_dismissed` logged. - `srs_modal_shown` logged once on open. - Options are non-interactive after the first tap (idempotent). - Dismissible by drag/tap-outside.

Dependencies: `analyticsServiceProvider` (FirebaseAnalytics), `dioClientProvider`, `sessionIdProvider`, `chatMessagesProvider.setSessionFeedback`. **Risks/unknowns:** If no `sessionId` is present, the rating is only kept in memory (never persisted). Only analytics-wired UI in the app.

MOBILE APP 22. Voice toggle

File: `lib/src/widgets/voice_toggle.dart`

User-visible behaviour A "Voice / Read messages aloud" switch that appears at the top of chat and in Settings. It is rendered **only if** the device reports the en-US TTS pack as available; otherwise it is hidden entirely. While the voice preference is loading it shows a spinner row. Toggling persists the choice and enables/disables `TtsService`.

Test points - Hidden when `isLanguageAvailable('en-US')` is false. - Loading → spinner row (no switch). - Toggling flips the switch and updates `voiceSettingsStateProvider`; `TtsService.setVoiceEnabled` called in lock-step. - Persisted via `PATCH /settings/voice`; failure is silent (logged) with optimistic UI retained. - Initial value loaded from `GET /settings/voice`.

Dependencies: `voiceSettingsProvider` (`GET` / `PATCH /settings/voice`), `ttsServiceProvider`. **Risks/unknowns:** Because it self-hides on missing en-US, the toggle can be invisible on devices lacking that pack (and `TtsService.speak` then silently no-ops). Optimistic toggle never rolls back on network failure.

MOBILE APP 23. Journal entry form

Files: `lib/src/features/journal/journal_screen.dart`, `journal_provider.dart`, `journal_entry.dart` · **Route:** `/journal`

User-visible behaviour A "Journal" screen with a dismissible privacy notice (#24), today's date/time, an emotion chooser (8 chips: Anxious/Sad/Angry/Overwhelmed/Numb/Calm/Hopeful/Other), a 1–5 intensity dot row, a multi-line "What's on your mind?" field (hint "Write anything — this is just for you."), Save / Cancel buttons and a "View History" link. Save is enabled only when an emotion is selected and text is non-empty. On success the free-text field is cleared and the app returns to `/home`. Text can be pre-filled from a chat "Save to journal" handoff.

Test points - Save disabled until emotion selected AND content non-empty. - Selecting a chip highlights it; intensity dots fill 1–5. - Save → `POST /journal {emotion,intensity,content}`; success → invalidate list, reset form, `/home`; failure → inline error "Network error. Please try again." / "Failed to save entry. Please try again.". - Cancel resets the form and returns Home. - Back arrow and system back both reset the form and return Home (`PopScope`). - Chat handoff prefill appears in the field. - Tapping outside the field dismisses the keyboard. - View History → `/journal/history`.

Dependencies: `journalEntryFormProvider`, `journalListProvider` (`GET /journal`), `dioClientProvider`, `POST /journal`, `SharedPreferences`. **Risks/unknowns:** Form provider is **not** autoDispose (deliberate, to survive the handoff); a `ref.listen` + explicit Save-handler both invalidate the list and navigate (double-path — check for double navigation). Success branch keeps text in the controller until the screen rebuilds.

MOBILE APP 24. Journal privacy notice

File: `lib/src/features/journal/journal_screen.dart`

User-visible behaviour A dismissible card: "Your journal is private. OKUVU doesn't read it and your therapist can't see it." with an . Dismissal is persisted locally so it stays hidden on subsequent visits.

Test points - Shown on first use; hiding it sets `journal_privacy_notice_dismissed = true` in SharedPreferences. - Hidden on next launch/visit once dismissed. - Dismiss button only hides the notice (does not clear form/navigate).

Dependencies: `shared_preferences` key `journal_privacy_notice_dismissed`. **Risks/unknowns:** Purely local; not synced to account, so it reappears on a fresh install or different device.

MOBILE APP 25. Journal history list

Files: `lib/src/features/journal/journal_list_screen.dart`, `journal_provider.dart`, `journal_entry.dart` · **Route:** `/journal/history`

User-visible behaviour A "Journal" list of past entries. Each card shows the timestamp, an emotion chip (colored per `emotionColors`), a 1–5 intensity dot row, and the content (2-line ellipsis, tap to expand). Loading shows a spinner; errors show "Failed to load entries." with Retry; an empty list shows "No entries yet. Your thoughts have a place here.".

Test points - Loading spinner → list of cards. - Error state → Retry re-fetches (`journalListProvider` invalidated). - Empty state copy. - Tapping a card expands/collapses the full text. - Timestamp formatting ("5 Jan 2026 · 3:04 PM"). - Emotion chip label/color maps; unknown emotion falls back to grey. - Back arrow / system back → `/home`.

Dependencies: `journalListProvider` → `GET /journal` (expects `{entries:[...]}`). **Risks/unknowns:** No pagination (loads all entries); `GET /journal` response must contain `entries` or parsing throws into the error state.

MOBILE APP 26. Progress screen (mood chart + care phase)

Files: `lib/src/features/progress/progress_screen.dart`, `providers/progress_provider.dart`, `models/mood_data_point.dart` · **Route:** `/progress`

User-visible behaviour "Your Progress" with a "Mood over the last 14 days" line chart (each mood logged is a dot; days without a check-in are gaps) plus the caption "Each dot is a mood you logged.". Above the chart is a "Your care journey" card with 5 phase dots. Loading → spinner; error → "Could not load your mood trend" + "Please check your connection and try again." + Retry; no data at all → "Start checking in to see your progress here." with the care-journey card.

Test points - 14-day window ends today; the x-axis weekday labels are correct. - Chart breaks the line at missing days (multiple `LineChartBarData` segments) and shows dots only for logged days. - Empty (all days null) → empty-state screen. - Error → Retry (`moodProgressProvider` invalidated). - `GET /progress/mood?days=14`; tolerates both `{entries:[...]}` and a raw list. - Care phase card shows the 5 labels (Getting to know each other / Building tools / Practising / Settling in / Maintaining). - Back arrow / system back → `/home`.

Dependencies: `moodProgressProvider` (`GET /progress/mood?days=14`), `userCarePlanProvider` (`GET /users/me/context`), `fl_chart`. **Risks/unknowns:** `CarePhaseCard`'s `isActive` is hard-coded `false`, so no phase ever highlights — the "care journey" is currently static/placeholder. `carePlanId` is passed but unused.

MOBILE APP 27. Settings hub

File: `lib/src/features/settings/settings_screen.dart` · **Route:** `/settings`

User-visible behaviour A "Settings" list: My Focus Area, Emergency Contact, Privacy & Data, Language, About, then the Voice toggle (#22) and Quote-of-the-Day toggle (#38), a divider, and a red "Delete Account" row. Footer shows "Version 1.0.0".

Test points - Each tile routes correctly: `/settings/focus`, `/settings/emergency`, `/settings/privacy`, `/settings/language`, `/settings/about`, `/settings/delete-account`. - Voice and Quote-of-Day toggles appear inline (subject to their own visibility rules). - Delete Account row uses the error color and routes to `/settings/delete-account`. - Version text present. - Back arrow / system back → `/home`.

Dependencies: child toggles/providers. **Risks/unknowns:** No links to the Profile or Notifications screens, even though routes exist (those screens are unreachable from the UI — see #33/#34). Version string ("1.0.0") disagrees with pubspec (1.0.4+5).

MOBILE APP 28. My Focus Area

File: `lib/src/features/settings/my_focus_area_screen.dart` · **Route:** `/settings/focus`

User-visible behaviour Shows the user's care plan. Loading → spinner; error → "Unable to load focus area"; no plan → "No focus area assigned yet"; otherwise a "Current care plan" card with the focus-area name and a status badge — **Provisional** (amber) when `assignment_state == 'provisional'`, otherwise **Confirmed** (green) — plus an info box explaining that the therapist sets this.

Test points - Loading/error/empty/populated states. - Badge text/color: `provisional` → Provisional/amber; anything else → Confirmed/green. - Focus-area name maps from CP1/CP2/CP3/other. - Info copy present. - Back → `/settings`.

Dependencies: `userCarePlanProvider` (`GET /users/me/context`). **Risks/unknowns:** Read-only; no way to change the plan in-app (by design). Relies on the shared care-plan provider.

MOBILE APP 29. Emergency Contact

Files: `lib/src/features/settings/emergency_contact_screen.dart`, `emergency_contact_notifier.dart` · **Route:** `/settings/emergency`

User-visible behaviour Two states. Empty/new: an explanatory card ("...encrypted and stored securely."), Name and Phone fields (phone hint `+1234567890`), a privacy note, and a Save button. Filled: a summary card showing the name and a **masked** phone (first char + asterisks + last 3 digits), an "Edit" button, and a "Remove Contact" button. Saving an empty form clears the contact.

Test points - Loading spinner while fetching; error message renders inline on failure. - Phone validation `^\+[1-9]\d{6,14}$` → invalid shows a snackbar "Phone must be in international format (e.g. +1234567890)". - Save (non-empty) → `PATCH /settings/emergency-contact {name,phone}`; UI switches to filled state. - Save with both fields empty → clears the contact. - Save disabled when nothing changed and no existing contact. - Remove Contact → `PATCH ...{clear:true}`, returns to the empty state and clears the fields. - Masked phone format correct (keeps last 3 digits). - Edit returns to the form pre-filled. - Back → `/settings`.

Dependencies: `emergencyContactProvider` (`GET` + `PATCH /settings/emergency-contact`). **Risks/unknowns:** Clearing via empty-save is easy to trigger accidentally; `hasContact` is defined as `name != null || phone != null`. Load failure defaults to empty state with an error string.

MOBILE APP 30. Privacy & Data (consent preferences)

Files: `lib/src/features/settings/privacy_data_screen.dart` , `consent_provider.dart` · **Route:** `/settings/privacy`

User-visible behaviour Consent toggles mirroring onboarding: Safety monitoring (Required, locked on), Mood understanding, Progress tracking, Emergency contact. A "Save Preferences" button is enabled only when something changed; a success/failure flows through the provider. Below a divider: "Download my data" (shows a "coming soon" snackbar), "Privacy Policy" (→ `/settings/privacy-policy`), "Delete my account" (→ `/settings/delete-account`).

Test points - Loading spinner while fetching consent; then current values from `engagement.consent_1..4` . - Safety monitoring switch locked (no `onChanged`) with "Required" badge. - Toggling a switch enables Save; Save disabled when unchanged. - Save → `POST /users/me/consent {safety_monitoring, mood_inference, assessment, emergency_contact}` ; success clears "changed" state; error rendered inline. - "Download my data" → snackbar "Data export is coming soon.". - Privacy Policy and Delete-account rows route correctly. - Back → `/settings` .

Dependencies: `consentProvider` (`GET /users/me/context` , `POST /users/me/consent`). **Risks/unknowns:** `_savedState` only becomes non-null after a successful fetch, so Save stays disabled if the initial fetch failed (error path). Uses the same `POST` as onboarding.

MOBILE APP 31. Language settings (English | Mirror)

Files: `lib/src/features/settings/language_screen.dart` , `language_notifier.dart` · **Route:** `/settings/language`

User-visible behaviour "How OKUVU talks with you" with two selectable rows: **English** ("OKUVU replies in English only.") and **Mirror** ("OKUVU mixes English and Hindi the way you do."). The selected row shows a filled check icon. Selecting updates immediately (optimistic) and persists in the background.

Test points - Loading spinner; then the current `language_mode` is reflected. - Selecting English/Mirror updates the check icon immediately. - Change persists via `PATCH /settings/language {language_mode}` ; failure is silent (logged) with optimistic UI retained. - Defaults to `english` when the backend omits the field or on fetch error. - Back → `/settings` .

Dependencies: `languageSettingsProvider` (`GET` / `PATCH /settings/language`). **Risks/unknowns:** Optimistic-only; no error surfaced to the user on a failed PATCH. Does not localize any client-side UI strings — it only tells the backend how to reply.

MOBILE APP 32. About

File: `lib/src/features/settings/about_screen.dart` · **Route:** `/settings/about`

User-visible behaviour Branding screen: a `FlutterLogo` (80), "OKUVU", "AI Mental Wellness Companion", "Version 1.0.0", an outlined "Privacy Policy" button (→ `/settings/privacy-policy`), and a "Back to Settings" button.

Test points - All texts render; version string present. - Privacy Policy button → `/settings/privacy-policy`. - Back to Settings → `/settings`; system back → `/settings`.

Dependencies: none beyond routing. **Risks/unknowns:** Uses the **stock Flutter logo** (no OKUVU logo asset); version "1.0.0" is hard-coded and mismatches pubspec.

MOBILE APP

33. Privacy Policy viewer

File: `lib/src/features/settings/privacy_screen.dart` · **Route:** `/settings/privacy-policy`

User-visible behaviour An informational screen with a privacy icon, "Privacy Policy", copy prompting the user to read the policy, and a "Read Privacy Policy" button that opens `AppConfig.privacyPolicyUrl` in an in-app web view. If the launch fails, a snackbar "Unable to open Privacy Policy." appears.

Test points - Opens the exact `AppConfig.privacyPolicyUrl` in `LaunchMode.inAppWebView`. - Launch failure → snackbar, no crash. - Back → `/settings`.

Dependencies: `url_launcher`, `AppConfig.privacyPolicyUrl`. **Risks/unknowns:** External Google-Docs dependency (needs connectivity; Docs may not render well in a webview).

MOBILE APP

34. Profile screen (stub)

File: `lib/src/features/settings/profile_screen.dart` · **Route:** `/settings/profile`

User-visible behaviour A minimal placeholder: person icon, "Profile Settings" heading, "Back to Settings" button. No editable fields.

Test points - Renders the placeholder; button → `/settings`; system back → `/settings`.

Dependencies: none. **Risks/unknowns: Unreachable from the UI** — no Settings row links to it; only reachable by typing the route. Not a real profile editor.

MOBILE APP

35. Notifications screen (stub)

File: `lib/src/features/settings/notifications_screen.dart` · **Route:** `/settings/notifications`

User-visible behaviour A placeholder: notifications icon, "Notification Settings" heading, "Back to Settings" button. No notification controls exist.

Test points - Renders the placeholder; button → `/settings`; system back → `/settings`.

Dependencies: none. **Risks/unknowns: Unreachable from the UI**; no push-notification permission/registration anywhere in the app (no Firebase Messaging dependency).

MOBILE APP

36. Account deletion

File: `lib/src/features/settings/account_deletion_screen.dart` · **Route:** `/settings/delete-account`

User-visible behaviour A multi-step flow with slide/fade transitions: **warning** ("Before you go" + a list of what will be deleted + "Continue to delete" / "Cancel"), **confirm** (a pulsing warning icon + "Delete my account permanently" / "I've changed my mind"), **processing** (spinner + "Deleting your account…"), and **success** ("Your account has been deleted." → auto sign-out and `/splash` after 3 s). A **sessionError** step ("Active session") appears if deletion is blocked by an active chat, offering "End session and continue" (→ `/chat`) or "Cancel". A 409 `ALREADY_DELETED` signs the user out to `/splash`.

Test points - Warning → Continue → Confirm; Cancel returns to Settings. - Confirm → `POST /account/delete {confirm:true}` → processing → success. - Success auto-signs-out and navigates to `/splash` after 3 s. - 409 `ACTIVE_SESSION` → sessionError step; "End session and continue" → `/chat`; "Cancel" → Settings. - 409 `ALREADY_DELETED` → signOut + `/splash`. - Any other error → back to Confirm with "Something went wrong. Please try again.". - Back button visibility per step (hidden on confirm/processing/success; shown on warning/sessionError). - System back on warning/sessionError → Settings; elsewhere → warning step. - Double-submit prevented (`isSubmitting`).

Dependencies: `dioClientProvider`, `POST /account/delete`, `authServiceProvider.signOut`. **Risks/unknowns:** The sessionError "End session and continue" navigates to `/chat` rather than actually closing the session/deleting — the user must manually end the chat then retry deletion. Terminal state relies on a 3 s timer (no manual dismiss).

MOBILE APP 37. Quote-of-the-Day toggle

Files: `lib/src/widgets/quote_of_day_toggle.dart`, `features/settings/quote_of_day_notifier.dart`

User-visible behaviour A "Quote of the Day / A short supportive line on your Home screen" switch (shown in Settings). Optimistic toggle persisted in the background; loading shows a spinner row.

Test points - Initial value from `GET /settings/quote-of-day {enabled}`; loading → spinner row. - Toggling updates immediately and persists via `PATCH /settings/quote-of-day {enabled}`; failure silent. - Does not itself render the quote (that is server-driven via `GET /users/me/today.quote`).

Dependencies: `quoteOfDaySettingsProvider` (`GET` / `PATCH /settings/quote-of-day`). **Risks/unknowns:** Optimistic-only, no error rollback. Effect depends on the backend honouring the preference in `/users/me/today`.

MOBILE APP 38. Crisis overlay

File: `lib/src/features/crisis/crisis_overlay.dart`

User-visible behaviour A full-screen, non-dismissible (system-back blocked) red (`0xFFBA1A1A`) overlay: a shield icon, "You're not alone.", "Here are some people you can call right now.", and five tappable helpline cards (iCall 9152987821, Vandrevala 18602662345, NIMHANS 08046110007, Snehi 04424640050, Emergency 112) each with a call button that dials via `tel:`. A subtle "I'm safe for now" link dismisses it. Shown app-wide via a `Stack` slot in `app.dart`, triggered by a `CRISIS_SIGNAL` WebSocket frame or the Home "Crisis Support" button.

Test points - All five resource cards render with correct names/descriptions/display numbers. - Each call button launches `tel:<number>` (url_launcher). - "I'm safe for now" hides the overlay (`crisisOverlayProvider.hide()`). - System back does not dismiss (crisis-aware dispatcher + `PopScope canPop:false`). - Chat back button is disabled while the overlay is up. - Overlay appears on `CRISIS_SIGNAL` regardless of current route.

Dependencies: `crisisOverlayProvider`, `webSocketServiceProvider.crisisSignalStream`, `url_launcher`. **Risks/unknowns:** Previously `hide()` was dead code (now wired) — regression-test the dismiss. Two hard-coded `Key`s exist for tests

(crisisOverlayShieldIcon , crisisOverlayDismissButton).

MOBILE APP 39. Not-found screen

File: lib/src/core/not_found_screen.dart

User-visible behaviour Shown by GoRouter.errorBuilder for unknown routes: a search-off icon, "Page not found", "The screen you're looking for doesn't exist.", and a "Go Home" button → /home .

Test points - Unknown path renders this screen (app bar "OKUVU"). - "Go Home" → /home .

Dependencies: go_router errorBuilder . **Risks/unknowns:** none.

MOBILE APP 40. TTS / Voice subsystem

Files: lib/src/core/tts_service.dart , lib/src/widgets/voice_toggle.dart , lib/src/features/settings/voice_settings_notifier.dart , plus the independent FlutterTts in guided_breathing_card.dart

User-visible behaviour When voice is enabled, agent chat replies are spoken aloud via TtsService.speak . Availability is gated on the **en-US** language pack; the guided-breathing card uses a **separate** FlutterTts instance set to **en-IN** and speaks its own instructions/counts. The voice preference is stored server-side and synced app-wide via voiceSyncProvider .

Test points - speak is a no-op when disabled. - When enabled but en-US is unavailable, speak silently returns (no crash). - Each incoming agent turn triggers speak(content) ; disabling voice calls tts.stop() . - voiceSyncProvider keeps TtsService in sync with the provider state. - Breathing card voice is independent and gated on the same voice-enabled setting. - Engine errors are caught and logged.

Dependencies: flutter_tts , voiceSettingsProvider (GET / PATCH /settings/voice), voiceSyncProvider .
Risks/unknowns: Two TTS stacks (service vs card) and mismatched locale gates (en-US vs en-IN); no user-facing feedback when a pack is missing. Android only declares the TTS_SERVICE query intent (no explicit TTS permission needed).

MOBILE APP 41. Language mode (English | Mirror) — data path

Files: language_notifier.dart , language_screen.dart , providers.dart (exports)

User-visible behaviour The user selects English or Mirror; the choice is persisted (PATCH /settings/language) and read back (GET /settings/language) to drive how the backend phrases replies. No client-side UI translation occurs.

Test points - Persistence round-trips across restarts. - Invalid/unknown backend values fall back to english . - Selection is reflected on re-entry to the screen.

Dependencies: GET / PATCH /settings/language . **Risks/unknowns:** Silent persistence failure (optimistic UI). No visible effect on any locally-rendered strings.

MOBILE APP 42. Theme / branding

Files: lib/src/app.dart , plus hard-coded 0xFF1A6B5A across features

User-visible behaviour Material 3 with a teal seed; brand teal `0xFF1A6B5A` used for primary buttons, agent avatars, charts and accents; mood colors `1→#E8A020, 2→#D4956A, 3→#6B6B6B, 4→#2D8A5E, 5→#1A6B5A`; crisis red `0xFFBA1A1A`. `fontFamily: 'Inter'` referenced in splash/fatal screen.

Test points - Consistent brand teal across screens. - Mood colors match the documented map. - Crisis overlay is the designated red, not themed. - Text renders with the intended font (or confirm fallback).

Dependencies: none. **Risks/unknowns:** No fonts declared in `pubspec.yaml` despite `fontFamily: 'Inter'` → the app will use the platform default font (branding gap). No dark theme (`ThemeMode` left default).

MOBILE APP

43. WebSocket protocol (shared contract)

File: `lib/src/core/websocket_service.dart` , `websocket_constants.dart`

Endpoint: `{wsBaseUrl}/ws` , `Authorization: Bearer <token>` header. Connect timeout 10 s. Reconnect with exponential backoff `1<<(n-1)` s, max **3** attempts, then `error` / `max_retries_exceeded`. Connection states: `disconnected, connecting, connected, sessionActive, reconnecting, error`.

Outbound frames - `SESSION_OPEN {payload:{session_id?}}` — sent on connect (and silently re-sent after an `IDLE_TIMEOUT` close). - `USER_TURN {payload:{session_id, message}}` — only when `sessionActive` and a session id exists. - `TOOL_RESULT {payload:{session_id, tool_type:'grounding_5431', result:{see,touch,hear,smell,taste}}}`. - `SESSION_CLOSE {}` — on intentional disconnect.

Inbound frames - `SESSION_OPEN_CONFIRMED {payload:{session_id}}` → state `sessionActive`, resets retry count. - `SESSION_OPEN_REJECTED {payload:{reason}}` → `error` with that reason (e.g. `rate_limited`). - `AGENT_TURN {payload:{message|content, turn_id, timestamp, turn_count, tool_card?, journal_prompt_suggested?}}` → agent turn stream. - `CRISIS_SIGNAL {payload:{session_id}}` → crisis stream (shows overlay). - `ERROR {payload:{reason}}` → logged only. - `SESSION_CLOSED {payload:{reason}}` → if `IDLE_TIMEOUT`, silently reopen a new thread and notify `IDLE_TIMEOUT`; otherwise set disconnected and emit the reason.

Test points - Correct headers/path; `SESSION_OPEN` sent immediately on connect. - Connect timeout forces `error(connect_timeout)` + reconnect. - Reconnect schedule 1s→2s→4s, stops at 3 attempts, banner shown after ~500 ms debounce and hidden on `sessionActive`. - `sendUserTurn` / `sendToolResult` no-op when not `sessionActive` or no session id. - `IDLE_TIMEOUT` close reopens a session and does **not** show the closed UI/SRS. - Other `SESSION_CLOSED` reasons set the closed UI + SRS. - `disconnect()` sends `SESSION_CLOSE` and suppresses reconnection (`_intentionalDisconnect`). - Unknown frame types are ignored without crashing; malformed JSON is caught.

Dependencies: `FlavorConfig.devBypassToken` / Firebase id token, `AppConfig.wsBaseUrl`. **Risks/unknowns:** `_hadError` / `_onDone` interplay can swallow a state transition after an error; `retry()` resets the attempt counter and reconnects. Inbound `payload` nesting for `AGENT_TURN` is assumed to match the model parser.

MOBILE APP

44. API surface (all HTTP endpoints found)

Method	Path	Caller
POST	`/auth/verify-otp`	OTP login
POST	`/users/me/consent`	Consent screen; Privacy & Data
GET	`/users/me/context`	Cold-start; consent load; My Focus Area; Progress
GET	`/users/me/today`	Today card
POST	`/users/me/homework/:id/complete`	Today card "Mark done"
GET	`/progress/mood?days=14`	Progress chart
POST	`/progress/mood/checkin`	Mood check-in
POST	`/journal`	Journal save
GET	`/journal`	Journal list
GET	`/settings/emergency-contact`	Emergency contact load
PATCH	`/settings/emergency-contact`	Emergency contact save/clear
GET	`/settings/voice` · PATCH `/settings/voice`	Voice toggle
GET	`/settings/language` · PATCH `/settings/language`	Language
GET	`/settings/quote-of-day` · PATCH `/settings/quote-of-day`	Quote toggle
PATCH	`/sessions/:id/srs`	SRS modal
POST	`/account/delete`	Account deletion

Cross-cutting HTTP test points: Bearer token attached to every request; 401 → sign-out; 503 → up to 3 retries honouring `Retry-After`; 10 s timeouts; JSON content type.

MOBILE APP 45. Local storage keys

- `shared_preferences`: `journal_privacy_notice_dismissed` (bool).
- No other local persistence found (auth tokens live in Firebase; preferences are server-side).

MOBILE APP 46. Firebase / platform integrations

- `firebase_core`, `firebase_auth` (phone OTP), `firebase_crashlytics` (fatal + platform errors), `firebase_analytics` (SRS events only).
- Android permissions/queries (`AndroidManifest.xml` — main): `INTERNET`; queries for `TTS_SERVICE` and `DIAL`. **No** microphone, camera, contacts, notification permissions.
- Flavors: dev (`.dev`), beta (`.beta`), prod — each sets only the app label in its manifest; `applicationId` suffixes in `android/app/build.gradle`.
- `flutter_tts`, `url_launcher` (tel + in-app webview) are the other native touchpoints.

Test points: Crashlytics receives fatal startup errors; analytics events fire from the SRS modal; tel: and webview launches work.

Consolidated risks & unknowns (test-plan focus)

1. **Care-plan selection is missing from intake** — the e2e tests (`care_plan_0` , "Anxiety support", "Low Mood") and the brief reference UI that does not exist in `lib/` . Those e2e flows are stale.
2. **Stale unit test** — `test/features/intake/intake_screen_test.dart` expects an initial assistant message and filled segments that the current notifier never produces.
3. **Grounding card shadowing** — `tool_card_widget.dart` 's local static `GroundingCard` shadows the interactive one imported from `grounding_card.dart` , so chat renders a non-interactive grounding card and `TOOL_RESULT` grounding frames are effectively unreachable from the app (though the interactive card passes its own unit test).
4. **Unreachable stubs** — `ProfileScreen` and `NotificationsScreen` are routed but have no UI entry point; no push-notification support exists.
5. **OTP existing-user dead-end** — if the backend reports `is_new_user == false` , no navigation occurs.
6. **Placeholders** — Home insight text, `ProgressSummary` (Home always `showData:false` , and the data state uses hard-coded "3"/"12"), progress care-phase (`isActive` always false), About/Flutter logo, and "Version 1.0.0" (vs pubspec 1.0.4+5).
7. **Fonts** — `fontFamily: 'Inter'` referenced but no font assets declared → default font.
8. **Optimistic settings** — voice/language/quote-of-day toggles never roll back or surface failures on PATCH errors.
9. **Consent Save locked on failed initial fetch** — `_savedState` stays null.
10. **TTS locale mismatch** — service gates on en-US while the breathing card uses en-IN; missing packs fail silently and hide the voice toggle.
11. **Intake reconnect wipes answers** — mid-questionnaire reconnect resets state to empty.
12. `/users/me/context` **error == no plan** on cold start (both route to consent/home), risking a wrong redirect on transient outages.
13. **Account-deletion sessionError path** routes to `/chat` instead of closing the session, so deletion requires a manual second attempt.
14. **Dead mock/legacy code** — `_useMockResponses=false` , commented-out `/crisis` navigation.
15. **No dark theme /** `ThemeMode` **default**; no localization of client strings despite a "Language" setting.

Count of user-facing features/screens documented above: 46 (incl. cross-cutting router/session, WS protocol, API surface, TTS, storage, Firebase, and the placeholder stubs).

- **Source:** `/home/shubham/okuvu-repo/apps/therapist-portal` (repo branch `develop`)
- **App:** Next.js 16 (App Router) + tRPC 11 + NextAuth 4 (Google OAuth) + Prisma 7 (`@okuvu/prisma`) + Tailwind 4
- **Scope of this document:** only what exists in code. Nothing here is aspirational.
- **Captured:** 2026-09-17
- **Counts:** 31 feature/page/component/infrastructure blocks + 19 tRPC procedures = **50 documented items**

> Notable absences (searched, not present): there is **no** `admin.createClient` procedure (the add-client flow is `admin.assignClient`); there is **no invite-token** mechanism and **no email delivery** anywhere in this app; there is **no client-facing consent UI** (consent is an admin/therapist attestation checkbox only); `PageHeader`, and the local `signIn` / `signOut` helpers in `lib/auth.ts`, are dead code.

A. Route map (App Router)

URL	File	Auth gate
`/`	`src/app/page.tsx`	proxy (server)
`/auth`	`src/app/auth/page.tsx`	public
`/dashboard`	`src/app/dashboard/page.tsx`	`auth()` + proxy
`/admin`	`src/app/admin/page.tsx`	`auth()` + `admin.amIAdmin`
`/clients/[id]`	`src/app/clients/[id]/page.tsx`	proxy + `clients.detail`
`/clients/[id]/program`	`src/app/clients/[id]/program/page.tsx`	proxy + `clients.getCarePlan`
`/clients/[id]/notes`	`src/app/clients/[id]/notes/page.tsx`	proxy + `notes.listNotes`
`/clients/[id]/homework`	`src/app/clients/[id]/homework/page.tsx`	proxy + `homework.listHomework`
`/clients/[id]/reminders`	`src/app/clients/[id]/reminders/page.tsx`	proxy + `reminders.listReminders`
`/clients/[id]/report`	`src/app/clients/[id]/report/page.tsx`	proxy + `reportCard.*`
`/api/auth/[...nextauth]`	`src/app/api/auth/[...nextauth]/route.ts`	public
`/api/trpc/[trpc]`	`src/app/api/trpc/[trpc]/route.ts`	per-procedure

Global layout: `src/app/layout.tsx` wraps everything in `TRPCProvider` → `AuthProvider`.

B. Auth & access-control infrastructure

THERAPIST PORTAL

B1. Root layout — `src/app/layout.tsx`

- **Route:** all routes (wrapper)
- **What it does:** Renders `<html><body>` with metadata (`OKUVU — Therapist Portal`), imports global CSS, and nests `<TRPCProvider>` → `<AuthProvider>` around `children`. No auth logic here.
- **Test points:** page title/description render; tRPC + session providers available on every route; no hydration errors from the two nested client providers; `<html lang="en">` present (a11y).
- **Dependencies:** `@/trpc/provider`, `@/lib/auth-provider`, `globals.css`.

- **Risks:** none material.

THERAPIST PORTAL **B2. Root redirect — `src/app/page.tsx`**

- **Route:** `/`
- **What it does:** Server component that unconditionally `redirect('/dashboard')`. No session check.
- **Test points:** unauthenticated `/` → 307 to `/dashboard` → proxy then bounces to `/auth`; authenticated `/` lands on dashboard; no redirect loop.
- **Dependencies:** Next `redirect`.
- **Risks:** hardcoded destination; unauthenticated users take two hops (`/` → `/dashboard` → `/auth`).

THERAPIST PORTAL **B3. Auth page — `src/app/auth/page.tsx`**

- **Route:** `/auth`
- **What it does:** Client component. On mount, if session is `authenticated` it `router.replace('/')`. Shows an OKUVU brand header, a Google sign-in button (`signIn('google')`), and contextual error panels keyed off `?error=`: `unauthorized` → "Your account is not yet set up on OKUVU..."; `server_error` → "A server error occurred. Please try again."; `OAuthSignin` / `OAuthCallback` → "Sign-in was cancelled. Please try again." While `status === 'loading'` it shows a spinner.
- **Test points:** spinner while loading; `?error=unauthorized` shows the allowlist message; `?error=server_error` and `?error=OAuthSignin` / `OAuthCallback` show their messages; a Google account already authenticated is redirected to `/`; button calls `signIn('google')`; the "Access is restricted to registered OKUVU therapists." footer renders; button is keyboard focusable and the SVG has no accessible-name issue (button text carries the label).
- **Dependencies:** `next-auth/react` (`useSession`, `signIn`), `useSearchParams` (wrapped in `<Suspense>`), Google provider configured (see B6).
- **Risks:** if `GOOGLE_CLIENT_ID` / `SECRET` are unset, `providers` is `[]` and the button silently fails (no provider configured) with no dedicated UI; `errorMessage` for unknown error codes is `null` (no fallback), so unrecognized errors render no panel; the wrong-`error` display is a soft client contract only.

THERAPIST PORTAL **B4. Auth middleware — `src/proxy.ts`**

- **Route:** everything matching `/((?!_next/static|_next/image|favicon.ico).*)`
- **What it does:** Server-side gate. Returns early (no gate) for `/api/auth*` and when `LOCAL_DEV_BYPASS_AUTH === 'true'`. Treats `/auth`, `/_next*`, `/favicon.ico` as public. If no `session.user` and route isn't public → redirect to `/auth`. If `session.user` exists and the route is `/auth` → redirect to `/`.
- **Test points:** unauthenticated deep link (e.g. `/clients/x`) → redirect to `/auth`; authenticated visit to `/auth` → redirect to `/`; `/api/auth/session` reachable while signed out; static assets not gated; bypass flag disables the gate entirely; matcher excludes `_next/static`, `_next/image`, `favicon.ico`.
- **Dependencies:** `getServerSession(authConfig)` from `@/lib/auth`; `LOCAL_DEV_BYPASS_AUTH`.
- **Risks:** the `/api/trpc/*` path is **not** in `PUBLIC_ROUTES` but is also not a page — proxy will redirect unauthenticated `/api/trpc` requests to `/auth` rather than letting tRPC return `UNAUTHORIZED` JSON; a bare authenticated user with no `Therapist` row passes proxy (the real allowlist enforcement is in the tRPC layer, see D1/D10).

THERAPIST PORTAL **B5. Production bypass boot guard — `instrumentation.ts`**

- **What it does:** Next.js `register()` hook (nodejs runtime only). Calls `checkProductionBypass`, which throws if `NODE_ENV === 'production'` and `LOCAL_DEV_BYPASS_AUTH === 'true'`. On throw it logs `FATAL: ...` and `process.exit(1)` so a misconfigured deploy fails the healthcheck rather than serving 500s.
- **Test points:** production + flag true → throws / process exits; production + unset/false → OK; dev/undefined + flag true → OK; the check runs before requests are served.

- **Dependencies:** env `NODE_ENV`, `LOCAL_DEV_BYPASS_AUTH`.
- **Risks:** relies on `NEXT_RUNTIME === 'nodejs'`; verified fail-closed (port stays open → 500s) but the explicit `process.exit(1)` is what makes deploy tooling actually fail.

THERAPIST PORTAL B6. NextAuth config + allowlist + `auth()` / `signIn` / `signOut` — `src/lib/auth.ts`

- **What it does:** Builds providers: a single `GoogleProvider` **only if** `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` are both set, else `[]`. Sets JWT session strategy with `maxAge = 12h` (F-13 clinical-data rationale). `signIn` callback: for Google + a profile email, if `LOCAL_DEV_BYPASS_AUTH === 'true'` allow; else dynamically import `prisma`, look up `therapist.findUnique({ where: { email } })`; found → allow; not found → `/auth?error=unauthorized`; no prisma → `/auth?error=server_error`; DB error → `/auth?error=server_error`. Non-Google attempts return `true` unchecked. `session` callback copies `token.sub` to `session.user.id`. `auth()` returns a mock session when the bypass flag is set, otherwise `getServerSession`. Exported `signIn` / `signOut` just `redirect('/auth')` (unused).
- **Test points:** Google email with no `Therapist` row → `?error=unauthorized`; with a row → success; non-Google provider passes through; missing/invalid DB → `?error=server_error`; session JWT `id` is populated; session expires after 12h idle; bypass flag returns the mock session (`id: 'mock'`, `email: 'bypass@okuvu.dev'`).
- **Dependencies:** `Therapist` table (email unique), `DATABASE_URL`, `GOOGLE_CLIENT_ID/SECRET`, `NEXTAUTH_SECRET` / `NEXTAUTH_URL`, `LOCAL_DEV_BYPASS_AUTH`.
- **Risks:** `maxAge` is 12h **JWT** — NextAuth re-issues on activity, so an active session can roll indefinitely; allowlist is a single email lookup with no domain/URI restriction config visible here; the unused `signIn` / `signOut` exports are a confusion trap; non-Google pass-through is only safe because no non-Google provider is configured.

THERAPIST PORTAL B7. Session provider — `src/lib/auth-provider.tsx`

- **What it does:** Client wrapper exporting a default component that renders NextAuth `SessionProvider` around children.
- **Test points:** `useSession()` resolves in the sidebar/top-bar; no provider nesting warnings.
- **Dependencies:** `next-auth/react`.
- **Risks:** none.

THERAPIST PORTAL B8. tRPC React provider/client — `src/trpc/provider.tsx`, `src/trpc/client.ts`

- **What it does:** `client.ts` exports `trpc = createTRPCReact<AppRouter>()`. `provider.tsx` creates a `QueryClient` and an `httpBatchLink` client pointed at `/api/trpc`, wiring `trpc.Provider` + `QueryClientProvider`.
- **Test points:** batched requests go to `/api/trpc`; query cache shared across pages; refetch on mutation success (pages call `refetch()`); no credentials/cookie misconfiguration (cookies ride along same-origin by default).
- **Dependencies:** `@tanstack/react-query`, `@trpc/client`.
- **Risks:** no auth headers passed client-side (relies on session cookie); no error/retry policy configured (defaults apply); no `httpBatchLink` URL env var (hardcoded `/api/trpc`).

THERAPIST PORTAL B9. Router composition — `src/server/routers/_app.ts`

- **What it does:** Assembles `appRouter` from `clients`, `notes`, `homework`, `reminders`, `reportCard`, `admin`. Exports `AppRouter` type.
- **Test points:** all six namespaces reachable at `/api/trpc/<router>.<procedure>`; type export consumed by the React client.
- **Dependencies:** these six router modules.
- **Risks:** none.

THERAPIST PORTAL B10. tRPC context, middlewares, auth/assignment helpers — `src/server/trpc.ts`

- **What it does:** Defines `createTRPCContext` (mock session when bypass else `auth()`; attaches `prisma`), `isAuthed` middleware → `protectedProcedure`, `therapistProcedure` (resolves the `Therapist` by session email, attaches `db` / `therapistId` / `therapistEmail`; in bypass uses `th_bypass_mock`; throws `UNAUTHORIZED` if not found), `therapistMutation` (= therapistProcedure + 20 req/60s per-therapist rate limit), `assertClientAssigned` (throws `FORBIDDEN` unless a `TherapistClient` row exists; no-op in bypass), and `logClinicalAccess` (best-effort `ClinicalAccessLog.create`, swallows failures, no-op in bypass).
- **Test points:** no session → `UNAUTHORIZED`; session without a `Therapist` row → `UNAUTHORIZED` ("Therapist not found"); unassigned client → `FORBIDDEN`; assigned → allowed; bypass bypasses both; access-log failure never breaks a read; mutation rate limit fires at 21 requests/60s; `therapistId` is the DB id (not the session id).
- **Dependencies:** `auth()`, `prisma`, `Therapist` / `TherapistClient` / `ClinicalAccessLog` tables.
- **Risks:** `ProtectedContext` / `isAuthed` allow `ctx.session.user` but `therapistProcedure` is the only place assignment identity is resolved — any future procedure that uses `protectedProcedure` directly (see D16 `admin.amIAdmin`) skips the Therapist lookup by design; rate-limit key is the email (see B11).

THERAPIST PORTAL B11. In-memory rate limiter — `src/server/rate-limit.ts`

- **What it does:** `checkRateLimit(key, max, windowMs)` keeps a `Map<string, number[]>` of timestamps, prunes older than the window, pushes now, returns `recent.length <= max`. Used as **`20 / 60_000`**.
- **Test points:** 20 allowed then 21st rejected (`TOO_MANY_REQUESTS`); window slides; separate keys independent; per-instance only.
- **Dependencies:** none (in-process memory).
- **Risks:** not shared across serverless instances (documented); `hits` map is never pruned of stale keys (unbounded growth over many distinct therapist emails); key is email string.

THERAPIST PORTAL B12. Server caller factory — `src/server/caller.ts`

- **What it does:** `createCaller(headers)` builds context via `createTRPCContext` and returns `appRouter.createCaller(ctx)` — used by the Vitest suites to call procedures directly.
- **Test points:** caller inherits session-based auth; used by `src/server/__tests__/*`.
- **Dependencies:** `_app`, `trpc`.
- **Risks:** test-only surface; no production use.

THERAPIST PORTAL B13. tRPC HTTP handler — `src/app/api/trpc/[trpc]/route.ts`

- **Route:** `/api/trpc/[trpc]`
- **What it does:** `fetchRequestHandler` with `endpoint: '/api/trpc'`, `appRouter`, and `createContext: () => createTRPCContext({ headers: req.headers })`; exports GET and POST.
- **Test points:** GET query batch works; POST mutation works; unauthenticated calls return tRPC `UNAUTHORIZED` (subject to B4 proxy ordering); malformed payload → tRPC error.
- **Dependencies:** `fetchRequestHandler`, `appRouter`, `createTRPCContext`.
- **Risks:** as noted in B4, proxy may intercept unauthenticated `/api/trpc` before this handler.

THERAPIST PORTAL B14. NextAuth route handler — `src/app/api/auth/[...nextauth]/route.ts`

- **Route:** `/api/auth/[...nextauth]`
- **What it does:** Re-exports `handlers.GET` / `POST` from `lib/auth.ts` (the `NextAuth(authConfig)` app).
- **Test points:** `/api/auth/signin`, `/api/auth/callback/google`, `/api/auth/session`, `/api/auth/signout` function; callback denial redirects to `/auth?error=unauthorized`.
- **Dependencies:** `NEXTAUTH_*`, `GOOGLE_*`.

- **Risks:** none beyond B6.

THERAPIST PORTAL **B15. `format.ts` helpers — `src/lib/format.ts`**

- **What it does:** `maskPhone(phone)` strips non-digits and returns `••• ••• <last4>` (returns the raw string if <4 digits). `formatClientLabel(id)` returns `User #<first4-of-id-uppercased>`.
- **Test points:** `+91 98765 43210` → last-4 mask; strings with <4 digits returned unchanged; ID with dashes stripped, first 4 chars uppercased; empty id → `User #`.
- **Dependencies:** none.
- **Risks:** `formatClientLabel('')` yields `User #`; masking is client-side only (server returns raw `phone_number` — see D1).

THERAPIST PORTAL **B16. Prisma singleton — `src/lib/prisma.ts`**

- **What it does:** Returns a cached `PrismaClient` (via `PrismaPg` adapter over a `pg.Pool`) or `null` when `DATABASE_URL` is missing/blank. Attaches a pool `error` handler to survive idle-connection drops (Neon).
- **Test points:** no `DATABASE_URL` → `prisma === null` → procedures throw `INTERNAL_SERVER_ERROR` ("Database not configured"); singleton reused across HMR; pool errors logged not crashed.
- **Dependencies:** `DATABASE_URL`, `@okuvu/prisma`, `pg`.
- **Risks:** `prisma` is evaluated at module load — env must be present at process start.

THERAPIST PORTAL **B17. Seed scripts — `src/scripts/seed-therapist.ts`, `seed-therapists-batch.ts`, `seed-client-assignments.ts`**

- **What it does:** CLI onboarding utilities. `seed-therapist.ts <email>` upserts one `Therapist`; `seed-therapists-batch.ts <email...>` upserts many; `seed-client-assignments.ts <email>=<userId>[,<userId>]` upserts `TherapistClient` rows (skipping unknown therapist/client). All load `.env` (or `apps/therapist-portal/.env`) and use `PrismaPg`.
- **Test points:** missing arg → usage error + exit 1; unknown therapist/client → SKIP log, no write; idempotent re-run; `DATABASE_URL` unset → clear error.
- **Dependencies:** `DATABASE_URL`, `Therapist` / `User` / `TherapistClient` tables.
- **Risks:** these create assignments **without** `chat_access_granted` (defaults false) — inconsistent with the admin flow that can set consent; no admin-email requirement.

C. Pages & components

THERAPIST PORTAL **C1. Dashboard page — `src/app/dashboard/page.tsx`**

- **Route:** `/dashboard`
- **What it does:** Server component. Awaits `auth()`; if no `session.user` → `redirect('/auth')`. Otherwise renders `<AuthenticatedShell>` wrapping `<DashboardContent therapistName={session.user.name ?? 'Therapist'} />`.
- **Test points:** unauthenticated hit redirects to `/auth`; authenticated renders shell + content; name falls back to "Therapist"; bypass flag yields the mock session (name "Bypass").
- **Dependencies:** `auth()`, `AuthenticatedShell`, `DashboardContent`.
- **Risks:** double auth check (proxy + page) is redundant but safe; page does not verify a `Therapist` row exists (allowlist enforced later at tRPC).

THERAPIST PORTAL **C2. Dashboard content — `src/app/dashboard/_components/DashboardContent.tsx`**

- **What it does:** Client component. `trpc.clients.list.useQuery()`. Shows a skeleton while loading, a red error card with a Retry button (calls `query.refetch()`) on error, and otherwise a time-based greeting (`Good morning/afternoon/evening, <name>.`) plus "You have N active client(s)." and `<ClientTable>`.

- **Test points:** loading skeleton (4 rows); error state message + Retry triggers refetch; greeting boundaries at 12:00 and 18:00 local time; singular/plural "client"/"clients"; zero clients shows the empty table state; greeting updates if hour changes on re-render.
- **Dependencies:** `clients.list`.
- **Risks:** greeting uses client clock/timezone; the word "active" is a UI label only — `clients.list` has no status filter.

THERAPIST PORTAL

C3. Client table — `src/app/dashboard/_components/ClientTable.tsx`

- **What it does:** Renders the caseload. Columns: **Client** (masked phone, links to `/clients/[id]`), **Care Plan** (badge mapping CP1→"Anxiety & Worry", CP2→"Low Mood", CP3→"Stress & Burnout"; `—` when null/unknown), **Status** (Provisional/Confirmed from `assignment_state`, `—` when no care plan), **Last Session** (`Today` / `Yesterday` / `N days ago` /<7d, else locale date; `—` when null), **Safety** (S3/S4 → red dot + "Review"; S2 → plain amber dot; otherwise blank), **Actions** ("View →" link). S3/S4 rows get a red left border + tinted background; other rows zebra-stripe. Empty state: "No clients assigned yet."
- **Test points:** masked phone shown, never the raw number; each CP badge; unknown CP → `—`; both status badges and `—` when carePlan null; last-session relative-date boundaries (today/yesterday/6d/ 7d/older); safety S0/S1 → blank, S2 dot, S3/S4 "Review" + row highlight; empty and single-row states; links navigate to the client; table header semantics; horizontal scroll on narrow viewports (`overflow-x-auto`); colour contrast of red/amber dots.
- **Dependencies:** `ClientListItem` type (`@okuvu/shared-types`), `maskPhone`.
- **Risks:** `isReviewRequired` is derived locally from `safetyState === 'S3' || 'S4'` and passed in, shadowing the `state` branch — S3/S4 never take the `state` path; `SafetyIndicator` renders nothing for S0/S1 or unknown/null (an empty cell, no accessible text); last-session math uses local midnight boundaries (`days` from raw ms), so "Today"/"Yesterday" can be off near midnight/DST.

THERAPIST PORTAL

C4. Authenticated shell — `src/app/components/AuthenticatedShell.tsx`

- **What it does:** Client wrapper used by `/dashboard` and `/admin`: renders `<Sidebar>`, a `md:ml-[240px]` content column, and a **mobile-only** top bar (`md:hidden`) with an OKUVU label and a Sign out button (`signOut({ callbackUrl: '/auth' })`).
- **Test points:** sign-out works on <md viewports; bar hidden at ≥md; content offset matches sidebar width; sign-out lands on `/auth`; keyboard/focus styles.
- **Dependencies:** `next-auth/react` `signOut`, `Sidebar`.
- **Risks:** this is one of three separate sign-out implementations (see C5, C6).

THERAPIST PORTAL

C5. Sidebar — `src/app/clients/[id]/_components/sidebar.tsx`

- **What it does:** Fixed 240px left nav (`hidden md:flex`). Links: **Dashboard** (active styling on `/dashboard`), a collapsible **Patients** section (only on `/clients/*`) that lists assigned clients by masked phone (from `clients.list`, enabled only on client pages) with an active/current highlight and loading/empty states, an **Admin** link (only when `admin.amIAdmin.isAdmin`), a **Reports** link (to the current client's `/report`; a disabled "role=link aria-disabled" placeholder when not on a client page), two disabled "Soon" items (Schedule, Safety Reviews), and a footer with the therapist avatar/initial + name and a **Sign out** button (`signOut({ callbackUrl: '/auth' })`).
- **Test points:** dashboard active state; Patients section only on client pages; client list uses masked phones; "No clients assigned" / "Loading…" states; current client highlighted; Admin link hidden for non-admins; Reports enabled only with a current client id; Soon items are not links and announce `aria-disabled`; sign-out; avatar falls back to initial with no image; `img` alt empty.
- **Dependencies:** `clients.list`, `admin.amIAdmin`, `useSession`, `maskPhone`.

- **Risks:** `params?.id` typed loosely (may be array); an `<img>` is used instead of `next/image`; Patients list re-fetches a full caseload query on client pages (no shared cache key with dashboard beyond tRPC dedupe); "Reports" is a `div` with `role="link"` when disabled (not focusable).

THERAPIST PORTAL

C6. Top bar — `src/app/clients/[id]/_components/top-bar.tsx`

- **What it does:** Sticky client-page header with a breadcrumb ("Dashboard" → client label, where the label comes from the parent layout's `clients.detail`), a mobile-only Sign out button (`signOut({ callbackUrl: '/auth' })`), and an avatar/initial.
- **Test points:** breadcrumb reflects the resolved client label; label absent while detail loads; sign-out on narrow viewports; sticky positioning; avatar fallback.
- **Dependencies:** `useSession`, `signOut`, `clientLabel` prop.
- **Risks:** another `<img>` avatar; the third sign-out copy.

THERAPIST PORTAL

C7. Client page header (unused) — `src/app/clients/[id]/_components/page-header.tsx`

- **What it does:** A presentational header with a phone number, an assignment-state dot, and two **disabled** buttons (Download Report, Log Interaction). No file imports it (dead code).
- **Test points:** none required at runtime; if revived — disabled buttons must be non-focusable and the state dot colour must match confirmed/provisional.
- **Dependencies:** `AssignmentState` type.
- **Risks:** dead code; disabled buttons use `cursor-not-allowed` but are still `<button disabled>` (acceptable).

THERAPIST PORTAL

C8. Admin page — `src/app/admin/page.tsx`

- **Route:** `/admin`
- **What it does:** Server component. `auth()` guard → `redirect('/auth')`; then renders `<AuthenticatedShell><AdminContent /></AuthenticatedShell>`.
- **Test points:** unauthenticated → `/auth`; authenticated but non-admin still loads the page and sees the in-page restriction panel (see C9); bypass session loads the page.
- **Dependencies:** `auth()`, `AdminContent`.
- **Risks:** page-level guard does not enforce admin (only tRPC/`AdminContent` do) — the route is reachable by any signed-in therapist, which is by design (restriction is rendered client-side).

THERAPIST PORTAL

C9. Admin content — `src/app/admin/_components/AdminContent.tsx`

- **What it does:** Client component. Queries `admin.amIAdmin`; if not admin, renders a red "This page is restricted to OKUVU admins." panel (with a hint to add the email to `ADMIN_EMAILS`). For admins: **Add a therapist** form (email required + optional name → `admin.addTherapist`), **Add client** form (therapist `<select>` from `admin.listTherapists`, phone required, optional language mode `english|mirror`, and a `chat_access_granted` consent checkbox → `admin.assignClient`), and a **Therapists** roster table (name/email, client count, createdAt) with loading/error/empty states. Success/error messages use `role=status` / `role=alert` + `aria-live`.
- **Test points:** non-admin sees the restriction panel and `listTherapists` is **not** fired (`enabled: amIAdmin.data?.isAdmin === true`); add-therapist rejects invalid email (native + Zod); optional name update path; assign-client succeeds and reports `wasAlreadyAssigned`; consent checkbox defaults unchecked; language select "Not set" omits the field; NOT_FOUND when no client matches the phone; forms clear on success; roster empty state; roster error state; busy/disabled states; label/input associations and live-region announcements.
- **Dependencies:** `admin.amIAdmin`, `admin.listTherapists`, `admin.addTherapist`, `admin.assignClient`; User.phone unique; `THERAPIST`, `ADMIN` Therapist.clientLabel; `ADMIN_EMAILS`.

- **Risks:** therapist `<select>` is populated only from `listTherapists` (empty until an admin exists); phone must match an existing `User.phone_number` exactly (no normalization server-side — a whitespace- trimmed but format-mismatched number yields NOT_FOUND); re-running assign never updates consent (documented); no way to unassign or change consent/language afterwards in this UI.

THERAPIST PORTAL C10. Client layout — `src/app/clients/[id]/layout.tsx`

- **Route:** `/clients/[id]/*`
- **What it does:** Client layout. Reads `params.id`, queries `clients.detail` (enabled when id present) to derive a `User #XXXX` label, then renders `<Sidebar>` + `<TopBar clientLabel>` + `<main>` children.
- **Test points:** label present once detail resolves; layout renders while detail loads/errors; sidebar
- top bar present on every client sub-route; unknown id still renders layout (detail error surfaces in the page).
- **Dependencies:** `clients.detail`, `formatClientLabel`.
- **Risks:** `clients.detail` is fetched here **and** again in each page (React Query dedupes, but the detail call performs an assignment check + writes a `ClinicalAccessLog` row for `VIEW_CLIENT_DETAIL` on every client page load).

THERAPIST PORTAL C11. Client overview page — `src/app/clients/[id]/page.tsx`

- **Route:** `/clients/[id]`
- **What it does:** Client component using `clients.detail`. Shows breadcrumb, **Client Overview** (masked phone + safety state with full `SAFETY_LABELS` — S0 Stable/S1 Low/S2 Elevated/S3 High/ S4 Critical, red for S3/S4), **Care Plan** card (plan name, assignment badge, an optional "Care plan may need review..." flag, a `<details>` explainer for provisional/confirmed, and a Manage → link to `/program`), **Clinical reasoning** card (therapist-only reasoning note or an empty-state explanation), **Latest session summary** card (parses JSON `summary_text` → `narrative_summary`; shows "generating" pulse or the no-summary state), and link cards to Homework, Reminders, Session Notes, and Report. Loading skeleton + error card.
- **Test points:** masked phone; each safety label/colour incl. null → "None"; care plan present/absent; provisional vs confirmed badges and explainer text; `careReviewFlagged` banner only when true; reasoning note present/absent; summary `available` / `generating` / `not_available_yet`; JSON vs non-JSON `summary_text` fallback; links route correctly; loading/error; assignment-denied (FORBIDDEN) renders the error card; breadcrumb link back to dashboard.
- **Dependencies:** `clients.detail`, `SAFETY_LABELS`, `maskPhone`, `formatClientLabel`.
- **Risks:** `summaryNarrative` trusts the first JSON key; the "Last session: {formatDateTime(session_id)}" line formats the **session id** as a date — a display bug (session_id is not a timestamp); reasoning note and summary text are **not** gated on `chat_access_granted` (only `careReviewFlagged` is) — the code comment explicitly flags this as a known inconsistency / potential content leak.

THERAPIST PORTAL C12. Program / care plan page — `src/app/clients/[id]/program/page.tsx`

- **Route:** `/clients/[id]/program`
- **What it does:** Client component. Loads `clients.getCarePlan`; shows the current plan (name + "Assigned on " + status badge). A `Select` of CP1/CP2/CP3; **Save care plan** is disabled until the selection differs; clicking Save opens an inline confirmation ("The AI companion will follow this track starting with their next session.") with Cancel / Confirm change → `clients.setCarePlan`; on success shows "Care plan saved." then redirects to `/clients/[id]` after 1.2s; on error shows the message. A closing notice explains notes aren't used by OKUVU.
- **Test points:** current plan/date/badge; Save disabled with no change; confirm dialog appears, Cancel returns, Confirm fires mutation; saving state; success message + redirect; error message stays on page; switching selection resets the confirm panel; loading/error; FORBIDDEN for unassigned client; invalid enum rejected (Zod); the AI-track warning copy is present.

- **Dependencies:** `clients.getCarePlan`, `clients.setCarePlan`; `ClinicalConfig` table.
- **Risks:** `setCarePlan` always writes `assignment_state: 'confirmed'`, so any change also confirms a provisional plan (possibly intended, but it couples two concepts); the page cannot clear a care plan; 1.2s redirect timer is not cancellable; the effect that syncs selection depends only on `care_plan_id` (a changing `assignment_state` alone won't resync).

THERAPIST PORTAL C13. Session notes page — `src/app/clients/[id]/notes/page.tsx`

- **Route:** `/clients/[id]/notes`
- **What it does:** Client component. Shows a privacy notice ("never read by the AI"). Lists notes from `notes.listNotes` (newest first, relative timestamps, optional therapist name). An add-note form with a `sessionStorage` draft (`note-draft-<clientId>` persisted on every keystroke), client-side non-empty validation, and `notes.addNote`; on success clears the draft, refetches, and shows "Note saved." for 3s; errors auto-dismiss after 5s.
- **Test points:** privacy copy; note list order/content/timestamps/author; loading + error + empty states; empty/whitespace submission → validation error and no call; textarea disabled while pending; draft survives a reload in the same tab and is cleared on success; success/error banners + `aria-live`; long/multi-line note wraps (`whitespace-pre-wrap`) and is escaped; FORBIDDEN for unassigned client.
- **Dependencies:** `notes.listNotes`, `notes.addNote`; `TherapistNote` table (therapist-scoped).
- **Risks:** notes are **per-therapist** (`where: { therapist_id }`) — a second assigned therapist will not see the first's notes, despite the UI implying a shared record; draft is `sessionStorage` only (lost on tab close); input is unbounded in length; `addNote` does not write an access log.

THERAPIST PORTAL C14. Homework page — `src/app/clients/[id]/homework/page.tsx`

- **Route:** `/clients/[id]/homework`
- **What it does:** Client component. Info notice ("Homework you assign here appears in the client's chat context"). Lists homework from `homework.listHomework` with a StatusBadge (Active/Completed/ Archived) and "Assigned "; active items get a **Mark completed** button → `homework.markCompleted`. Assign form with `sessionStorage` draft (`homework-draft-<clientId>`), non-empty validation, → `homework.assign`; success clears draft + refetches + 3s toast; errors dismiss after 5s.
- **Test points:** list order/status badges/relative times; Mark completed only on `active`; pending disable; empty/whitespace validation; draft persistence; success/error banners; empty state; loading and error states; FORBIDDEN; ownership check — marking a row belonging to another therapist/client returns NOT_FOUND (not a silent success).
- **Dependencies:** `homework.listHomework`, `homework.assign`, `homework.markCompleted`; `Homework` table (`HomeworkStatus`).
- **Risks:** status filter is per-therapist in the list; the "Archived" badge is unreachable from this UI (no archive action); `markCompleted` is one-way (no un-complete); `assign` doesn't log access.

THERAPIST PORTAL C15. Reminders page — `src/app/clients/[id]/reminders/page.tsx`

- **Route:** `/clients/[id]/reminders`
- **What it does:** Client component. Info notice ("Push delivery isn't wired up yet — the client sees these when they open the app."). Lists reminders from `reminders.listReminders` (schedule note, Active/Paused badge, frequency label, `HH:mm` times, tool label) with a **Pause/Resume** toggle → `reminders.setActive`. **New reminder** form: schedule text (required client-side), frequency select (`off|once_daily|twice_daily|thrice_daily|as_needed`), optional tool select (from `reminders.listTools`), and `type=time` inputs whose count follows the frequency (1/2/3); submit → `reminders.create` with success/error banners.

- **Test points:** list rendering incl. times/tool; Pause↔Resume; frequency change resizes the times array (preserving entered values, defaulting `09:00`); `off` / `as_needed` send `times: []`; custom times sent only for `*_daily`; validation of `HH:mm` server-side; `.max(3)`; empty schedule note blocks submit; success clears note + refetches (frequency/tool/times are **not** reset — verify intended); empty/loading states; FORBIDDEN; setActive NOT_FOUND when the reminder belongs to another client; missing `listTools` data leaves only "None".
- **Dependencies:** `reminders.listReminders`, `reminders.listTools`, `reminders.create`, `reminders.setActive`; `Reminder`, `ToolRegistryEntry` tables.
- **Risks:** `times` regex allows an empty array but `create` accepts up to 3 arbitrary valid `HH:mm` — no server check that the count matches the frequency; reminders are `where: { user_id }` (not therapist-scoped) so any assigned therapist sees all of a client's reminders; no unassigned-client guard on `listTools` (it's global data), `homeworkId` is supported by the API but no UI sets it; `setActive` has no `onError` handler (failures are silent).

THERAPIST PORTAL **C16. Report / Session Prep page — `src/app/clients/[id]/report/page.tsx`**

- **Route:** `/clients/[id]/report`
- **What it does:** Client component. **Session Prep** card from `reportCard.sessionPrep`: amber "Client hasn't granted chat access yet." when `granted:false`, else the digest of the last 30 days' summaries (date, narrative summary, themes, open loops) or an empty state. **Report Card** card from `reportCard.reportCard`: amber not-granted state, else S-07 signal chips (taper state, closure offered, frustration count, therapist referral), "Homework assigned / completed" counts, a homework track list, and a mood-trend sparkline of 30-day mood values (with `title` tooltips).
- **Test points:** both cards' `granted:false` states; empty Session Prep (no summaries in 30d); each signal chip appears only when set; counts rendering (`active+completed / completed`); homework-track list only when non-empty; mood cells with tooltip and "No mood check-ins…" empty state; loading skeletons; FORBIDDEN; consent gating means no clinical query is made when not granted.
- **Dependencies:** `reportCard.sessionPrep` (`sessionSummary`), `reportCard.reportCard` (`sessionSummary`, `homework`, `userMemory`, `moodLog`, `turn`).
- **Risks:** the "assigned" figure is `active + completed` and ignores `archived` rows (can under-report and contradicts the label); mood cell shows the raw numeric `mood_value` with only a hover title (poor for touch/screen-reader users); `entries.map` keys on `sessionId` (collision if a session has multiple summaries) and mood cells key on `date`; both queries fire in parallel so a FORBIDDEN error surfaces per-card, not page-wide.

D. tRPC procedures (19 total)

All procedures live under `/api/trpc`. Unless noted, they are gated by `therapistProcedure` (authenticated **and** a `Therapist` row must exist) and mutations add the 20/60s rate limit.

THERAPIST PORTAL **D1. `clients.list` — query — `src/server/routers/clients.ts`**

- **Auth:** `therapistProcedure`
- **What it does:** Finds all `TherapistClient` rows for the therapist, including each client's `clinical_config`, `safety_state`, and the single most recent `session_logs` row. Returns `{ id, phoneNumber (raw), carePlan: {care_plan_id, assignment_state} | null, lastSessionDate (ISO) | null, safetyState | null }`.
- **Test points:** only assigned clients returned; ordering (DB default — unordered); `carePlan` null when no `ClinicalConfig`; `lastSessionDate` null when no session logs, else the newest; `safetyState` null when no row; empty caseload → `[]`; unauthenticated → UNAUTHORIZED; unknown therapist → UNAUTHORIZED.
- **Dependencies:** `TherapistClient`, `User`, `ClinicalConfig`, `SafetyState`, `SessionLog`.

- **Risks:** returns the **unmasked** `phone_number` (masking is client-side only); **no** `ClinicalAccessLog` entry is written for a list read; no pagination or ordering.

THERAPIST PORTAL **D2. `clients.detail` — query — `src/server/routers/clients.ts`**

- **Input:** `{ clientUserId: string }`
- **Auth:** `therapistProcedure` + `assertClientAssigned` + `logClinicalAccess('VIEW_CLIENT_DETAIL')` (bypass returns a fixed stub instead).
- **What it does:** Resolves the `User` with `clinical_config`, `safety_state`, and the newest `session_summaries` (with session). Derives `summaryStatus` (`available` when COMPLETED **and** non-null text; `generating` when PENDING; otherwise `not_available_yet`). Returns `{ id, phoneNumber, carePlan, reasoningNote, safetyState, summary: {status, summary_text, session_id}, careReviewFlagged }`, where `careReviewFlagged = chat_access_granted && care_plan_review_flagged`.
- **Test points:** FORBIDDEN for unassigned; NOT_FOUND for a missing user (note: assignment is checked first); access-log row written with `VIEW_CLIENT_DETAIL`; each `summaryStatus` mapping incl. FAILED/SKIPPED_TRIVIAL/no-summary; `careReviewFlagged` true only when consent is granted (both directions); bypass stub shape; `reasoningNote` / `summary_text` returned regardless of consent.
- **Dependencies:** `TherapistClient`, `User`, `ClinicalConfig`, `SafetyState`, `SessionSummary`, `SessionLog`, `ClinicalAccessLog`.
- **Risks:** reasoning note + summary text are not consent-gated (documented known inconsistency); `assertClientAssigned` runs before the user lookup so a nonexistent id yields FORBIDDEN for unassigned callers and NOT_FOUND only for assigned ones.

THERAPIST PORTAL **D3. `clients.getCarePlan` — query — `src/server/routers/clients.ts`**

- **Input:** `{ clientId: string >=1 }`
- **Auth:** `therapistProcedure` + `assertClientAssigned`
- **What it does:** Reads `ClinicalConfig` by `user_id`, defaulting to `{care_plan_id:'CP1', assignment_state:'provisional', created_at:null}` when no row exists.
- **Test points:** existing config returned with ISO `created_at`; missing config → CP1/provisional/null; FORBIDDEN for unassigned; empty-string id rejected (Zod); bypass → stub (assignment check no-ops).
- **Dependencies:** `ClinicalConfig`.
- **Risks:** synthesizes a default that looks like a real assignment — a client with no config appears as "CP1 / Provisional"; no access log.

THERAPIST PORTAL **D4. `clients.setCarePlan` — mutation — `src/server/routers/clients.ts`**

- **Input:** `{ clientId: string >=1, carePlanId: 'CP1'|'CP2'|'CP3' }`
- **Auth:** `therapistMutation` (rate-limited) + `assertClientAssigned`
- **What it does:** Upserts `ClinicalConfig` for the client, setting `care_plan_id` and forcing `assignment_state: 'confirmed'`. Returns `{ success, clientId, carePlanId, assignmentState, created_at }`.
- **Test points:** create path when no config exists; update path when it does; `assignment_state` becomes `confirmed` in both; invalid `carePlanId` rejected; FORBIDDEN for unassigned; rate limit at 21/60s; empty `clientId` rejected; bypass no-op assignment check + upsert still runs.
- **Dependencies:** `ClinicalConfig`.
- **Risks:** always flips provisional→confirmed (coupling); no access log; no optimistic-concurrency guard (last write wins).

THERAPIST PORTAL **D5. `notes.addNote` — mutation — `src/server/routers/notes.ts`**

- **Input:** `{ clientUserId: string, noteText: string (trimmed, min 1) }`

- **Auth:** `therapistMutation` + `assertClientAssigned`
- **What it does:** Creates a `TherapistNote` for (therapist, client) and returns `{ id, created_at }`.
- **Test points:** happy path persists trimmed text; empty/whitespace-only rejected with "Note text must not be empty"; FORBIDDEN for unassigned; rate limiting; very long text accepted (no max — confirm intended); bypass.
- **Dependencies:** `TherapistNote`.
- **Risks:** no max length; no access log; notes are therapist-private (per D6), which may surprise users.

THERAPIST PORTAL **D6. `notes.listNotes` — query — `src/server/routers/notes.ts`**

- **Input:** `{ clientId: string >=1 }`
- **Auth:** `therapistProcedure` + `assertClientAssigned` + `logClinicalAccess('VIEW_NOTES')`
- **What it does:** Returns the calling therapist's notes for the client, newest first, with `{ id, note_text, created_at, therapist.name }`.
- **Test points:** only the caller's notes (a co-assigned therapist's notes are hidden); ordering desc; access log written with `VIEW_NOTES`; empty → `[]`; FORBIDDEN; bypass.
- **Dependencies:** `TherapistNote`, `Therapist`, `ClinicalAccessLog`.
- **Risks:** per-therapist scoping vs. the UI's implied shared record; `therapist.name` may be null.

THERAPIST PORTAL **D7. `homework.assign` — mutation — `src/server/routers/homework.ts`**

- **Input:** `{ clientUserId: string, description: string (trimmed, min 1) }`
- **Auth:** `therapistMutation` + `assertClientAssigned`
- **What it does:** Creates `Homework` (status defaults `active`) and returns `{ id, status, assigned_at }`.
- **Test points:** happy path; empty/whitespace rejected; FORBIDDEN; rate limit; bypass; the description becomes the client's chat context (integration point with `apps/api`).
- **Dependencies:** `Homework`.
- **Risks:** no access log; no max length; no duplicate/active-item guard (multiple active items — downstream `agent-context.ts` picks the most recent `active` row).

THERAPIST PORTAL **D8. `homework.listHomework` — query — `src/server/routers/homework.ts`**

- **Input:** `{ clientId: string >=1 }`
- **Auth:** `therapistProcedure` + `assertClientAssigned` + `logClinicalAccess('VIEW_HOMEWORK')`
- **What it does:** Returns the calling therapist's homework for the client, newest first: `{ id, description, status, assigned_at, updated_at }`.
- **Test points:** therapist-scoped rows; desc ordering; access log `VIEW_HOMEWORK`; empty; FORBIDDEN.
- **Dependencies:** `Homework`, `ClinicalAccessLog`.
- **Risks:** a co-assigned therapist sees a different homework list than the client's chat context (which reads by client, not therapist) — divergence risk.

THERAPIST PORTAL **D9. `homework.markCompleted` — mutation — `src/server/routers/homework.ts`**

- **Input:** `{ homeworkId: string, clientId: string >=1 }`
- **Auth:** `therapistMutation` + `assertClientAssigned` + ownership check
- **What it does:** Loads the homework row and throws NOT_FOUND unless it belongs to both this therapist **and** the claimed client; otherwise sets `status: 'completed'` and returns `{ id, status, updated_at }`.
- **Test points:** success; NOT_FOUND when the row belongs to another therapist; NOT_FOUND when the row belongs to another client; NOT_FOUND when the row does not exist; FORBIDDEN when the caller isn't assigned to the claimed client; rate limit.

- **Dependencies:** `Homework`.
- **Risks:** idempotent-ish (re-marking completed is a no-op update); no access log; no un-complete.

THERAPIST PORTAL **D10. `reminders.listReminders` — query — `src/server/routers/reminders.ts`**

- **Input:** `{ clientId: string >=1 }`
- **Auth:** `therapistProcedure` + `assertClientAssigned` + `logClinicalAccess('VIEW_REMINDERS')` (bypass returns `[]`).
- **What it does:** Returns all `Reminder` rows for the **user** (not therapist-scoped), newest first, with the optional tool `{toolId,label}`: `{id, scheduleNote, frequency, times, active, homeworkId, tool, createdAt}`.
- **Test points:** all client reminders (regardless of creator) returned; tool joined or null; access log `VIEW_REMINDERS`; empty; FORBIDDEN; bypass `[]`.
- **Dependencies:** `Reminder`, `ToolRegistryEntry`, `ClinicalAccessLog`.
- **Risks:** not therapist-scoped (any assigned therapist sees/controls all reminders); no `homeworkId` surfaced in the UI although the API returns it.

THERAPIST PORTAL **D11. `reminders.listTools` — query — `src/server/routers/reminders.ts`**

- **Input:** none
- **Auth:** `therapistProcedure` (no per-client gate — static reference data)
- **What it does:** Returns all `ToolRegistryEntry` rows, label-ascending, mapped to `{toolId, label, chatGuidable}`.
- **Test points:** returns the seeded tool ids; ordering; accessible without a client context; no access log; unauthenticated → UNAUTHORIZED.
- **Dependencies:** `ToolRegistryEntry`.
- **Risks:** seeds only 3 tool ids today (per schema comment) — the select can be sparse.

THERAPIST PORTAL **D12. `reminders.create` — mutation — `src/server/routers/reminders.ts`**

- **Input:** `{ clientUserId: string >=1, scheduleNote: string (trimmed,min1), frequency: 'off'|'once_daily'|'twice_daily'|'thrice_daily'|'as_needed', times: string[] (each /^([01]\d|2[0-3]):([0-5]\d)$/, max 3), homeworkId?, toolId? }`
- **Auth:** `therapistMutation` + `assertClientAssigned` + `logClinicalAccess('CREATE_REMINDER')`
- **What it does:** Creates a `Reminder` and returns `{id, active, created_at}`.
- **Test points:** happy path per frequency; `times` validation (rejects `9:00`, `24:00`, `09:60`); `.max(3)` enforced; 4+ times rejected; `homeworkId` / `toolId` optional persistence; empty note rejected; FORBIDDEN; rate limit; access log `CREATE_REMINDER`.
- **Dependencies:** `Reminder`, optional `Homework` / `ToolRegistryEntry` FKs.
- **Risks:** no server-side check that `times.length` matches the frequency; a bogus `toolId` / `homeworkId` surfaces as a raw FK error, not a friendly validation message; no access-log on failure paths.

THERAPIST PORTAL **D13. `reminders.setActive` — mutation — `src/server/routers/reminders.ts`**

- **Input:** `{ id: string, clientUserId: string >=1, active: boolean }`
- **Auth:** `therapistMutation` + `assertClientAssigned` + ownership check
- **What it does:** Loads the reminder, throws NOT_FOUND unless `reminder.user_id === clientUserId`, else updates `active` and returns `{id, active}`.
- **Test points:** pause and resume; NOT_FOUND when the reminder belongs to a different client; NOT_FOUND when the id doesn't exist; FORBIDDEN when unassigned to the claimed client; rate limit.
- **Dependencies:** `Reminder`.
- **Risks:** the ownership check trains on `user_id` only (no therapist scoping — intentional since reminders are client-owned); no access log.

THERAPIST PORTAL **D14. reportCard.sessionPrep — query — src/server/routers/report-card.ts**

- **Input:** { clientId: string >=1, days: int 1..365 = 30 }
- **Auth:** therapistProcedure + assertClientAssigned + **chat-consent gate** + logClinicalAccess ('VIEW_SESSION_PREP') (bypass returns {granted:true, entries:[]}).
- **What it does:** If chat_access_granted is false → {granted:false} (no clinical query). Else fetches COMPLETED session_summaries since now – days , newest first, parses each JSON summary_text (defensive fallback to {narrative_summary: raw}) and returns {granted:true, entries:[{sessionId, date, narrativeSummary, themes[], openLoops[]}]} .
- **Test points:** granted:false makes **no** sessionSummary query and writes no access log; granted:true writes VIEW_SESSION_PREP and maps entries; JSON vs non-JSON/legacy text fallback; missing keys default (themes / openLoops → []); days boundary (0 rejected, 366 rejected, 365 ok, default 30 when omitted); only COMPLETED summaries included; FORBIDDEN for unassigned; bypass stub.
- **Dependencies:** TherapistClient.chat_access_granted , SessionSummary , ClinicalAccessLog .
- **Risks:** consent defaults false and no client-facing UI sets it, so in practice this may always return "not granted" until consent capture lands (explicitly noted in code); summary text is parsed as a spec-less JSON blob.

THERAPIST PORTAL **D15. reportCard.reportCard — query — src/server/routers/report-card.ts**

- **Input:** { clientId: string >=1 }
- **Auth:** therapistProcedure + assertClientAssigned + **chat-consent gate** + logClinicalAccess('VIEW_REPORT_CARD') (bypass returns a stub with granted:true and empty data).
- **What it does:** If consent false → {granted:false} . Else, in parallel: last 5 COMPLETED summaries, homework.groupBy(status) counts, userMemory.homework_track , 30-day moodLog s (ascending), and 30-day assistant turn s for S-07 signals. Returns {granted, homeworkProgress, homeworkCounts:{active,completed}, homeworkTrack, moodTrend, signals:{latestTaperState, closureOfferedRecently, frustrationEventCount, therapistReferralRecently}} . latestTaperState is the newest turn's value (default 'none'); closureOfferedRecently / therapistReferralRecently are some(...) ; frustrationEventCount is the count.
- **Test points:** granted:false → no queries + no access log; granted:true → access log VIEW_REPORT_CARD ; counts map only active / completed (archived ignored); signals computed correctly for mixed turn sets; latestTaperState defaults to none with no turns; homeworkTrack defaults [] ; mood trend sorted ascending; homeworkProgress filters entries with null progress; FORBIDDEN; bypass stub shape.
- **Dependencies:** TherapistClient , SessionSummary , Homework , UserMemory , MoodLog , Turn , ClinicalAccessLog .
- **Risks:** homeworkCounts ignores archived , which makes the UI's "assigned" total (active + completed) inconsistent; moodLog query windows on date_ist compared against a JS-computed timestamp (timezone-sensitive); this is a read-only snapshot with no decisioning (per code comment).

THERAPIST PORTAL **D16. admin.amIAdmin — query — src/server/routers/admin.ts**

- **Input:** none
- **Auth:** protectedProcedure (any authenticated session — **no** Therapist row required)
- **What it does:** Returns { isAdmin: boolean } by checking ctx.user.email against the comma-separated, trimmed, lowercased ADMIN_EMAILS set.
- **Test points:** admin email (case-insensitive, surrounding whitespace tolerated) → true; non-admin → false; unauthenticated → UNAUTHORIZED; empty/unset ADMIN_EMAILS → false for everyone; bypass session email bypass@okuvu.dev only true if explicitly allowlisted.
- **Dependencies:** env ADMIN_EMAILS .
- **Risks:** makes the admin link discoverable to any signed-in user (by design); no role table — admin status is a pure env change; relies on session email being trustworthy (Google-verified).

THERAPIST PORTAL **D17. admin.listTherapists — query — src/server/routers/admin.ts**

- **Auth:** adminProcedure (protectedProcedure + admin-email check + requires prisma)
- **What it does:** Returns all therapists newest-first: {id, email, name, createdAt (ISO), clientCount} (count from client_assignments).
- **Test points:** FORBIDDEN for non-admin; requires only an allowlisted session email (no Therapist row needed); clientCount accuracy; empty roster → [] ; ordering desc; database not configured → INTERNAL_SERVER_ERROR.
- **Dependencies:** Therapist , TherapistClient .
- **Risks:** an allowlisted admin who is not a Therapist can still administer (intended); returns all therapist emails (PII) — restricted to the allowlist only.

THERAPIST PORTAL **D18. admin.addTherapist — mutation — src/server/routers/admin.ts**

- **Input:** { email: trimmed, email-valid; name?: trimmed min1 }
- **Auth:** adminMutation (adminProcedure + 20/60s rate limit keyed admin:<email>)
- **What it does:** Upserts a Therapist by email (updates name only when provided), returns {id, email, name} .
- **Test points:** creates a new therapist (the allowlist entry enabling Google sign-in); re-running updates the name; omitting the name leaves it unchanged; invalid email rejected; whitespace trimmed; FORBIDDEN for non-admin; rate limit at 21/60s; idempotent.
- **Dependencies:** Therapist ; interaction with the sign-in allowlist (B6).
- **Risks:** creating a therapist row is sufficient to grant portal access (no invite/verification step); no audit row is written for onboarding (unlike ClinicalAccessLog for reads).

THERAPIST PORTAL **D19. admin.assignClient — mutation — src/server/routers/admin.ts**

- **Input:** { therapistId: min1, clientPhoneNumber: trimmed min1, chatAccessGranted: boolean, languageMode?: 'english'|'mirror' }
- **Auth:** adminMutation (admin-only + rate limit)
- **What it does:** Looks up a User by exact phone_number (NOT_FOUND if none) and the Therapist by id (NOT_FOUND if none); checks for an existing TherapistClient ; upserts with update: {} (so a repeat never overwrites/revokes consent); optionally sets User.language_mode ; returns {success, therapistEmail, clientId, wasAlreadyAssigned} .
- **Test points:** new assignment writes chat_access_granted from input; repeat assignment → wasAlreadyAssigned: true and **no** consent change; languageMode provided → user.update called; omitted → not called; NOT_FOUND for unknown phone; NOT_FOUND for unknown therapist; FORBIDDEN for non-admin; rate limit; phone must match exactly (formatting/whitespace normalization gaps); consent checkbox false is a valid write.
- **Dependencies:** User.phone_number , Therapist , TherapistClient , User.language_mode .
- **Risks:** there is no way to change consent, unassign, or switch therapist after first assignment via this API; languageMode is overwritten on **every** call even when the assignment already exists (while consent is not) — an inconsistency; no audit row.

E. Cross-cutting behaviours

E1. LOCAL_DEV_BYPASS_AUTH bypass mode

When LOCAL_DEV_BYPASS_AUTH === 'true' (only permitted outside production — see B5): - proxy.ts skips all gating (B4). - createPrContext injects a mock session {id:'mock', email:'bypass@okuvu.dev', name:'Bypass'} (B10). - auth() returns the same mock session, so /dashboard and /admin don't bounce to /auth (B6). - The sign-in callback

short-circuits to `true` before the allowlist lookup (B6). - `therapistProcedure` falls back to `therapistId: 'th_bypass_mock'` when no `Therapist` row matches (B10). - `assertClientAssigned` and `logClinicalAccess` become no-ops (B10). - Several procedures return fixed stubs instead of querying: `clients.detail` (D2), `reminders.listReminders` → `[]` (D10), `reportCard.sessionPrep` → `{granted:true, entries:[]}` (D14), `reportCard.reportCard` → stub with `granted:true` (D15). - **Test points:** with the flag on — no `/auth` redirect anywhere; stubbed payloads match their documented shapes; mutations still hit the DB where implemented (`setCarePlan` , `addNote` , `assign` , `create` , `markCompleted` , `setActive`) — verify which ones still expect a database; boot guard refuses the flag in production; the flag is never baked into the client bundle (not in `next.config` env). - **Risks:** bypass does **not** stub `clients.list` / `getCarePlan` , so those still require a real DB; inconsistent stub coverage between reads and writes is an easy source of confusing test results.

E2. Rate limiting

- Applies to every `therapistMutation` (20 per 60s per therapist **email**) and every `adminMutation` (20 per 60s per `admin:<email>`). Reads are unlimited. In-memory per instance (serverless caveat).
- **Test points:** the 21st mutation in a minute → `TOO_MANY_REQUESTS` ; reads never throttled; keys are per-email; the limit applies across different procedures for the same user.
- **Risks:** trivially bypassed across instances; stale keys never evicted.

E3. Clinical access logging (F-05)

- `ClinicalAccessLog` rows are written for: `VIEW_CLIENT_DETAIL` , `VIEW_NOTES` , `VIEW_HOMEWORK` , `VIEW_REMINDERS` , `CREATE_REMINDER` , `VIEW_SESSION_PREP` , `VIEW_REPORT_CARD` . No-ops in bypass.
- **Test points:** each of the above writes exactly one row with the right `action` ; failures are swallowed (a broken log never blocks the read); reads that are **not** logged: `clients.list` , `getCarePlan` , `listTools` , and all mutation write-paths except `CREATE_REMINDER` .
- **Risks:** `clients.list` (a full caseload read incl. safety state) is unlogged; homework/notes writes are unlogged; logging is best-effort so a forensics gap is possible under DB errors.

E4. Consent gating (`TherapistClient.chat_access_granted`)

- Gates `reportCard.sessionPrep` , `reportCard.reportCard` , and `clients.detail.careReviewFlagged` . Defaults `false` . Only producer is `admin.assignClient` 's checkbox; **no client-facing consent UI exists**.
- **Test points:** with consent false — both report procedures return `granted:false` and issue no clinical queries/access logs; `careReviewFlagged` hidden; with consent true — data flows. Re-running assign does not flip consent.
- **Risks:** with no client-facing capture flow, this is effectively always false → report cards appear "not granted" to everyone in practice; `reasoningNote` / `summary_text` on the client detail page leak past this gate (documented).

F. Data dependencies (Prisma models touched by this app)

Defined in `packages/prisma/schema.prisma` ; this app references:

Model	Used by
`User` (`phone_number` , `language_mode`)	assignClient, clients.list/detail, all client-scoped pages
`Therapist` (`email` unique, `name` , `image`)	sign-in allowlist, therapistProcedure, admin roster
`TherapistClient` (`chat_access_granted` , unique `[therapist_id, client_user_id]`)	assignment checks, consent gate, admin assign
`ClinicalConfig` (`care_plan_id` , `assignment_state` , `reasoning_note` , `care_plan_review_flagged`)	clients detail/getCarePlan/setCarePlan
`SafetyState` (`safety_state_current`)	dashboard + client detail
`SessionLog` (`session_started_at`)	dashboard last-session
`SessionSummary` (`summary_text` , `status` , `session_id`)	client detail + report/session prep
`TherapistNote`	notes page/router
`Homework` (`status`)	homework page/router, report counts
`Reminder` , `ToolRegistryEntry`	reminders page/router
`Turn` (`taper_state` , `closure_offered` , `frustration_event` , `therapist_referral`)	report signals
`UserMemory` (`homework_track`)	report card
`MoodLog` (`mood_value` , `date_ist`)	report mood trend
`ClinicalAccessLog`	access logging

Enums: `CarePlan {CP1,CP2,CP3}` , `AssignmentState {provisional,confirmed}` , `SafetyStateEnum {S0..S4}` , `SummaryStatus {PENDING,COMPLETED,FAILED,SKIPPED_TRIVIAL}` , `HomeworkStatus {active,completed,archived}` , `LanguageMode {english,mirror}` .

G. Configuration / external integrations

- **Env vars:** `DATABASE_URL` , `NEXTAUTH_SECRET` , `NEXTAUTH_URL` , `GOOGLE_CLIENT_ID` , `GOOGLE_CLIENT_SECRET` , `ADMIN_EMAILS` , `LOCAL_DEV_BYPASS_AUTH` . (Firebase vars exist in `.env.example` but are **not read** by any therapist-portal source file.)
- **External services:** Google OAuth (sign-in), Postgres/Neon (data).
- **Email delivery:** none. **Invite tokens:** none.
- **Client → server surface:** `/api/trpc` (batched) and `/api/auth/*` .

H. Consolidated risks / unknowns

1. `clients.detail` **consent inconsistency** — `reasoningNote` and `summary_text` are returned without `chat_access_granted` ; only `careReviewFlagged` is gated (code comment admits this).
2. **Report-card "assigned" count ignores** `archived` — label can misreport totals.
3. **Client detail session date bug** — the no-summary branch formats `summary.session_id` via a date formatter.
4. **Notes are per-therapist, not shared** — UI implies a shared clinical record; a co-assigned therapist sees an empty history.

5. **Reminders and homework are read/written inconsistently scoped** — reminders by client, homework list by therapist, while the client's chat context reads homework by client → divergence.
6. `setCarePlan` **always confirms** — changing a plan silently promotes provisional→confirmed.
7. **Consent has no client-facing capture** — report features are effectively dead until it lands.
8. `admin.assignClient` **cannot revise consent/unassign** and overwrites `language_mode` on repeats while deliberately not touching consent.
9. **Rate limiting is per-instance in-memory** and its key map is never evicted.
10. `clients.list` **and key write-paths are not access-logged** — forensic gaps.
11. **Proxy may intercept unauthenticated** `/api/trpc` with an HTML redirect instead of JSON.
12. **Multiple divergent sign-out implementations** (shell, sidebar, top-bar) and **dead code** (`page-header.tsx` , unused `signIn` / `signOut` in `lib/auth.ts`).
13. `formatClientLabel(")` → `User #` ; masking is client-side only (raw phone leaves the server).
14. `ADMIN_EMAILS` **absent from the root** `.env.example` (only present in the app-level one) — easy to miss on a new environment.
15. **No pagination/ordering** on `clients.list` ; very large caseloads may render slowly.

5. Known failures, risks & blocked items (baseline — 17 Sep 2026)

Testing starts from a *known* state. Re-verify each item as it is fixed.

5.1 Test-suite baseline (Test Pass 1)

`flutter analyze` → **0 errors**, 2 warnings, 6 info. `flutter test` → **126 pass / 24 fail**. The failures cluster into four causes:

Cause	Failing	Notes
Firebase not initialised in tests (`No Firebase App '[DEFAULT]'`)	~20	intake (10) · chat (5) · home (3) · widget_test (1). Tests don't override `authServiceProvider` / `webSocketServiceProvider` . **Onboarding→home→chat spine has no green coverage.**
Uncancelled timer + `pumpAndSettle` timeouts	3	consent screen — the pending timer is a real code smell
Stale test expectations	2	`app_config_test` asserts the placeholder `evia-500215-xxxxx-uc.a.run.app` vs actual `backend-971397352159...`
`pumpAndSettle` timeout	1	auth→home navigation

5.2 Blocked (cannot be executed on the current box)

Blocker	Impact
No Android SDK / JDK / `/dev/kvm` ; no `adb` device	**L3 + L5 not runnable** (integration + manual device passes)
Firebase config absent (`google-services.json` , `GoogleService-Info.plist` , `firebase_options.dart`)	Cannot build a runnable app or exercise real auth
Beta/prod backend `backend-971397352159.asia-south1.run.app` → **404** on `/` , `/health` , `/api/health`	End-to-end runs against beta unverified — confirm the URL/service

5.3 MOBILE APP — risks flagged from code

1. Care-plan selection referenced by e2e tests/brief **does not exist** in intake → those e2e flows are stale.
2. Stale intake unit test (expects an initial message + 2 filled segments; code starts empty).
3. `GroundingCard` symbol shadowing in `tool_card_widget.dart` → chat renders the **non-interactive** card; the 5-4-3-2-1 flow (and its `TOOL_RESULT` frame) is unreachable in-app.
4. `ProfileScreen` / `NotificationsScreen` are routed but **unreachable stubs**; no push support.
5. OTP existing-user (`is_new_user == false`) **dead-end** — no navigation.
6. Placeholders: home insight text, `ProgressSummary` , care-phase `isActive` always false, Flutter logo, version string mismatch (1.0.0 vs 1.0.4+5).
7. `Inter` font referenced, **no font assets declared** → platform default.
8. Settings toggles are **optimistic with no rollback** on PATCH failure.
9. Consent "Save" locked if the initial fetch fails (`_savedState` stays null).
10. TTS locale mismatch (service `en-US` vs breathing card `en-IN`); missing packs fail silently.
11. Intake reconnect **wipes answers**; `/users/me/context` error treated as "no plan".

12. Account-deletion session-error path routes to `/chat` instead of closing.
13. Dead/legacy mock code; no dark theme; no client-string localisation.

5.4 THERAPIST PORTAL — risks flagged from code

1. `clients.detail` returns `reasoningNote` + `summary_text` **without consent gating** (only `careReviewFlagged` is gated) — privacy-inconsistent.
2. Report-card "assigned" count **ignores** `archived`.
3. Client-detail session-date bug — formats `summary.session_id` through a date formatter (can throw on a UUID).
4. Notes are **per-therapist, not shared**; a co-assigned therapist sees an empty history.
5. Reminders/homework scoping is **inconsistent** (by client vs by therapist) vs the client's chat context.
6. `setCarePlan` always confirms — silently promotes provisional → confirmed.
7. **No client-facing consent capture** → report features effectively dead until it lands.
8. `admin.assignClient` cannot revise consent/unassign; overwrites `language_mode` on repeats.
9. Rate limiting is **per-instance in-memory**; key map never evicted.
10. `clients.list` and key write paths are **not access-logged** (forensic gap).
11. Proxy may intercept unauthenticated `/api/trpc` with an HTML redirect instead of JSON.
12. Divergent sign-out implementations + dead code (`page-header.tsx` , unused `signIn` / `signOut`).
13. `formatClientLabel('') → 'User #'` ; masking is **client-side only** (raw phone leaves the server).
14. `ADMIN_EMAILS` missing from the root `.env.example` .
15. No pagination/ordering on `clients.list` .

---

6. Execution record

Copy this table per run (one row per surface section); attach evidence links.

Run date	Surface	Area	Level (L1–L7)	Tester	Pass	Fail	Blocked	Notes
	Mobile App	Part A						
	Therapist Portal	Part B						
	Both	§2 Cross-cutting						
	Mobile App	§5.3 crisis / safety						

End of register. Generated from code at `develop` `b4ee3cd` ; re-generate after any change to `apps/mobile` or `apps/therapist-portal` .